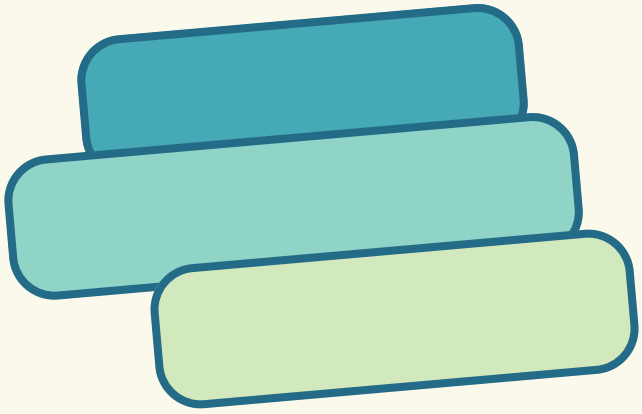
THE HANDWRITTEN TECH LIBRARY

VOL. 03



DOCKER EDITION

DOCKER

Zero to Hero

THE PREMIUM HANDWRITTEN INTERVIEW NOTEBOOK

From the first container to production architecture.

110 exact questions

110 answered follow-ups

110 mini diagrams

12 practice labs

JAVA & SPRING BOOT SCENARIOS / COMMAND-FIRST EXPLANATIONS / PRACTICAL WARNINGS

01 / HOW TO USE THIS BOOK

Learn Docker like you would explain it on a whiteboard.

Every original question is preserved word-for-word. Each answer is reorganized into a quick explanation, an example, a flow diagram, a real-world warning, and a **fully answered follow-up.**

110	22	12
original questions	chapters	labs

The five-step interview habit

01	Say it simply	Read the concise answer aloud.
02	Show the path	Explain the diagram left to right.
03	Point to evidence	Use the command or Java example.
04	Describe the risk	Show what could break in production.
05	Handle the follow-up	Answer the harder question briefly.

> **Lab safety:** The scripts are designed for disposable practice locations. Some commands delete only lab-created containers/volumes. Never replace those names with real production names.

Verification boundary: extracted Java sample compiled with JDK 17; all 12 Bash scripts passed shell syntax checks. Docker Engine is unavailable here, so container operations were not run or benchmarked.

02 / ROADMAP

All chapters

Foundation to senior production diagnosis. Every chapter has five original interview questions.

01	Docker foundations	Q001–Q005
02	Engine architecture and Linux isolation	Q006–Q010
03	Container lifecycle and essential commands	Q011–Q015
04	Images, tags, digests and registries	Q016–Q020
05	Dockerfile instructions and build context	Q021–Q025
06	ENTRYPOINT, CMD, PID 1 and signals	Q026–Q030
07	BuildKit, caching and build performance	Q031–Q035
08	Multi-stage builds and Java packaging	Q036–Q040
09	Persistent storage and mount behavior	Q041–Q045
10	Networks, DNS and published ports	Q046–Q050
11	Docker Compose applications	Q051–Q055

BONUS: complete Java example, 12 lab scripts, reference cards, source directory.

02 / ROADMAP

All chapters

Foundation to senior production diagnosis. Every chapter has five original interview questions.

12	Configuration and secrets	Q056–Q060
13	Health checks, restart policies and shutdown	Q061–Q065
14	CPU, memory and JVM container behavior	Q066–Q070
15	Logs, inspection and troubleshooting	Q071–Q075
16	Runtime security and least privilege	Q076–Q080
17	Image maintenance and supply-chain evidence	Q081–Q085
18	CI/CD and release delivery	Q086–Q090
19	Multi-platform images and native compatibility	Q091–Q095
20	Databases, Kafka and stateful workloads	Q096–Q100
21	Orchestration, scaling and operational boundaries	Q101–Q105
22	Senior interview scenarios	Q106–Q110

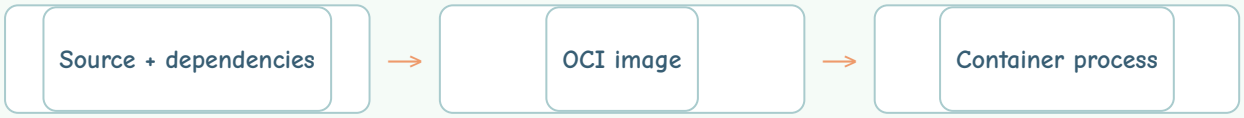
BONUS: complete Java example, 12 lab scripts, reference cards, source directory.

03 / THE BIG PICTURE

Four drawings worth memorizing

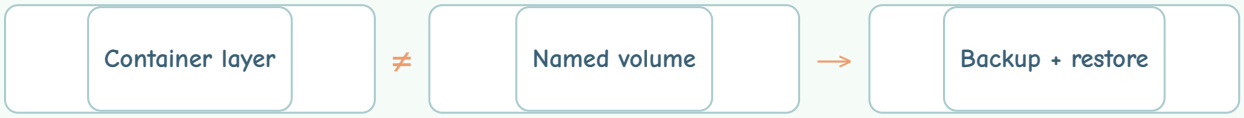
Docker becomes easier when you separate build-time, runtime, networking and recovery.

1. Build once → Run anywhere compatible



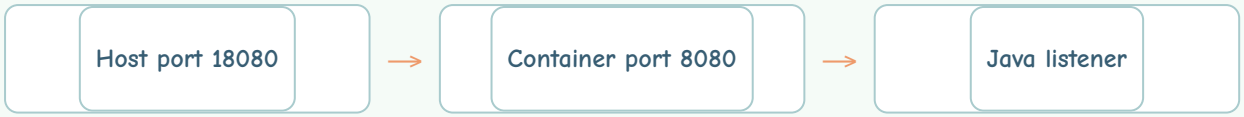
An image packages user-space files. It still needs the right OS/CPU environment.

2. Where data lives



A volume outlives the container. It does not automatically give off-host recovery.

3. Container networking



EXPOSE is documentation. A published port and active application listener are separate facts.

4. Safer Java shutdown



Runtime shutdown timing must allow Spring, database and worker cleanup to finish.

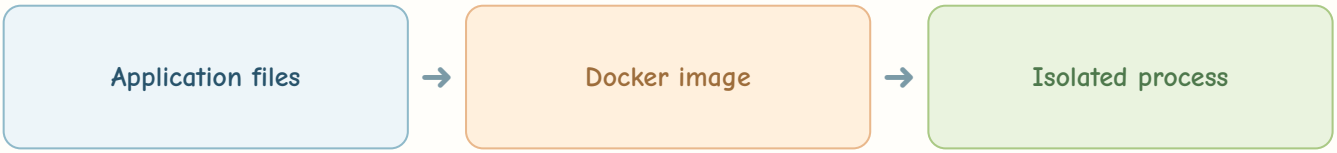
FOUNDATION

Q001 What is Docker, and what problem does it solve?

THE 20-SECOND ANSWER

Docker packages an application and its user-space dependencies into an image, then runs that image as an isolated process environment.

↳ Draw this in the interview



WHY / HOW IT WORKS

A service can use the same image in test and production while receiving a different database URL at startup. Packaging removes many installation differences; it does not remove network, permission or configuration differences.

COMMAND OR MINI EXAMPLE — from original guide

```
Developer: Java 21 + app.jar + OS libraries
Image:     the same packaged runtime and application
Server:    starts that image with its own configuration
```

THE INTERVIEWER MAY ASK…

Which production problems remain even when the image is identical?

A. The image does not control network access, database availability, secrets, CPU/memory limits, file permissions, or deployment configuration. Check these separately in production.

⚠ WATCH OUT — A container image does not include its own running kernel. External services still need compatible contracts and credentials.

ORIGINAL QUESTION PRESERVED

Source guide p. 6

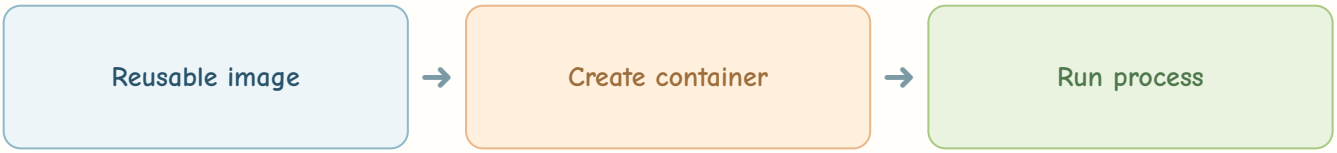
FOUNDATION

Q002 What is the difference between an image and a container?

THE 20-SECOND ANSWER

An image is a reusable template; a container is a created runtime instance with configuration, process state and a writable filesystem layer.

↳ Draw this in the interview



WHY / HOW IT WORKS

Pulling downloads an image. Creating records a container without starting its main process. Starting launches that process. Several containers can use one image and have separate writable layers.

COMMAND OR MINI EXAMPLE — from original guide

```
docker pull nginx:stable
docker create --name dk-q-web nginx:stable
docker start dk-q-web
docker inspect dk-q-web
```

THE INTERVIEWER MAY ASK…

What survives a restart, and what disappears when the container is removed?

A. A restart keeps the container object's configuration and writable layer. Removing the container removes its writable layer; named volumes and images normally remain unless explicitly removed.

⚠ WATCH OUT — Editing a running container does not modify the source image or the other containers created from it.

ORIGINAL QUESTION PRESERVED

Source guide p. 7

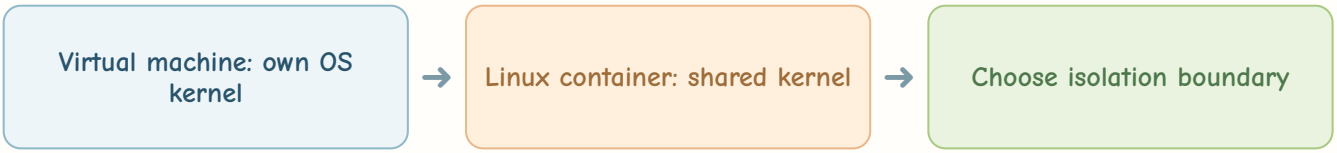
FOUNDATION

Q003 How are containers different from virtual machines?

THE 20-SECOND ANSWER

A conventional VM has a guest kernel; Linux containers isolate processes that share the kernel of the Linux host or Docker Desktop VM.

↳ Draw this in the interview



WHY / HOW IT WORKS

Containers commonly start faster because they launch processes rather than boot a full guest OS. A VM can run a different guest OS; a Linux image requires a compatible Linux execution environment.

COMMAND OR MINI EXAMPLE — from original guide

```
VM workload:         app + libraries + guest kernel
Linux container:     app + libraries
Shared foundation:   Linux host or Desktop VM kernel
```

THE INTERVIEWER MAY ASK...

When would you combine VMs and containers?

A. Run containers inside VMs when you need stronger host isolation, different tenant security boundaries, or cloud VM management combined with fast container deployment.

⚠ WATCH OUT — Shared-kernel isolation has a different threat boundary from a separate guest kernel. Startup and memory savings depend on the workload.

ORIGINAL QUESTION PRESERVED

Source guide p. 7

FOUNDATION

Q004 Why can a Docker image run on different computers?

THE 20-SECOND ANSWER

It supplies a consistent user-space filesystem, but portability still depends on OS, CPU architecture, kernel features and runtime configuration.

↳ Draw this in the interview



WHY / HOW IT WORKS

A Linux image on macOS or Windows usually runs through Docker Desktop's Linux VM. Multi-platform images can supply separate variants for AMD64 and ARM64. Native libraries inside a Java image must match the selected platform.

COMMAND OR MINI EXAMPLE — from original guide

```
docker image inspect IMAGE \
  --format '{{.Os}}/{{.Architecture}}'
# Example platform: linux/amd64
# An ARM laptop needs an ARM image or supported emulation.
```

THE INTERVIEWER MAY ASK…

Why can a JNI library work on one image variant and fail on another?

A. JNI contains CPU- and OS-specific native binaries. An amd64 Linux .so cannot automatically load on arm64 or another OS; package and test the right binary for each platform.

⚠ WATCH OUT — Windows containers use Windows compatibility rules. Java bytecode portability does not make every image architecture-neutral.

ORIGINAL QUESTION PRESERVED

Source guide p. 8

FOUNDATION

Q005 What are Docker Engine, Docker Desktop, Compose and a registry?

THE 20-SECOND ANSWER

Engine manages container objects; Desktop provides a workstation environment; Compose describes a multi-container application; a registry distributes images.

↳ Draw this in the interview



WHY / HOW IT WORKS

Compose uses the Docker API to create services, networks and volumes from configuration. A registry can store images without running them. Desktop adds integration around the engine for supported desktop systems.

COMMAND OR MINI EXAMPLE — from original guide

```
docker version          # client and engine information
docker compose version  # Compose plugin information
docker pull REGISTRY/TEAM/SERVICE:TAG
```

THE INTERVIEWER MAY ASK…

How would you confirm which engine receives your commands?

A. Use `docker context show`, `docker context inspect`, and `docker info`. Check the daemon endpoint and engine identity before running builds or deleting containers.

⚠ WATCH OUT — Installing the CLI alone does not prove that a daemon is running or that your selected context points to the intended machine.

ORIGINAL QUESTION PRESERVED

Source guide p. 8

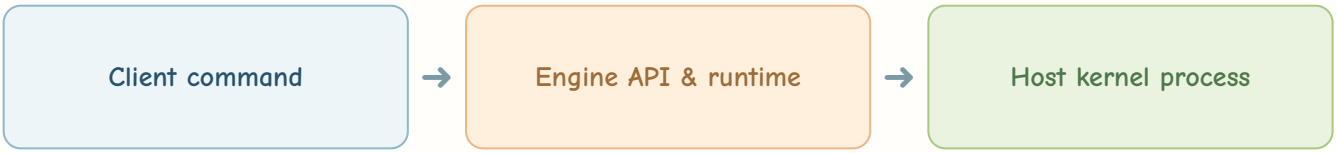
FOUNDATION

Q006 How does a Docker command reach a container process?

THE 20-SECOND ANSWER

The CLI calls the selected engine API; the engine coordinates container creation and execution through lower-level runtime components.

↳ Draw this in the interview



WHY / HOW IT WORKS

The terminal runs on the client machine, but containers and volumes live on the engine's machine. A remote context can therefore change where data is stored and where published ports appear.

COMMAND OR MINI EXAMPLE — from original guide

```
docker context show
docker context ls
docker info
# The selected context can address a remote engine.
```

THE INTERVIEWER MAY ASK…

Where does a bind mount come from when the engine is remote?

A. A bind mount path is resolved on the Docker Engine host, not necessarily on your local CLI computer. With Docker Desktop, check its VM/file-sharing behavior.

⚠ WATCH OUT — A client-side file path is not automatically a path on a remote daemon host. Check context before troubleshooting mounts or cleanup.

ORIGINAL QUESTION PRESERVED

Source guide p. 9

FOUNDATION

Q007 What do Linux namespaces do?

THE 20-SECOND ANSWER

Namespaces give processes separate views of resources such as process IDs, network interfaces and mounts.

↳ Draw this in the interview



WHY / HOW IT WORKS

The process may see a small process tree and its own hostname while still being a process on the host kernel. Namespaces change visibility and association; they do not allocate CPU or memory budgets.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --rm alpine:3.22 \
  sh -c 'echo "container view"; ps; hostname'
# Compare this view with the Linux host's process list.
```

THE INTERVIEWER MAY ASK…

How is namespace isolation different from resource limiting?

A. Namespaces control which resources a process can see; cgroups limit how much CPU/memory it may use. Isolation and resource budgets are different controls.

⚠ WATCH OUT — Explicit host namespace options reduce that isolation. Namespace separation alone is not a complete security boundary.

ORIGINAL QUESTION PRESERVED

Source guide p. 10

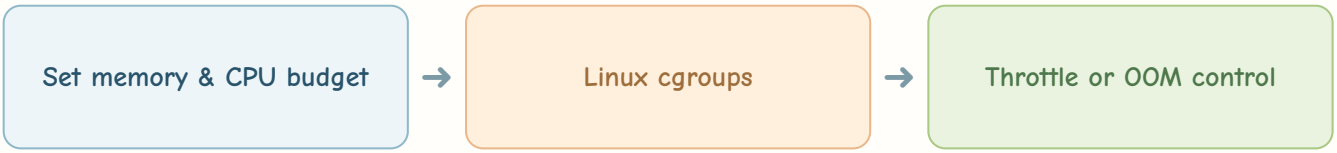
FOUNDATION

Q008 What are cgroups, and why do containers need them?

THE 20-SECOND ANSWER

Linux control groups account for and constrain resources consumed by groups of processes.

↳ Draw this in the interview



WHY / HOW IT WORKS

The process receives a memory ceiling and CPU bandwidth allowance. This keeps a runaway workload from consuming unrestricted resources, provided the host and runtime support the relevant controls.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --rm --memory=256m --cpus=0.5 \
  alpine:3.22 sleep 10
```

THE INTERVIEWER MAY ASK…

What happens when a process exceeds its memory limit?

A. The Linux memory controller can kill a process in the container; Docker may report OOMKilled. Inspect the memory limit, exit details, and host events.

⚠ WATCH OUT — A CPU quota is not a dedicated physical core. Container limits also do not reserve sufficient cluster capacity by themselves.

ORIGINAL QUESTION PRESERVED

Source guide p. 10

FOUNDATION

Q009 What are containerd, runc and OCI?

THE 20-SECOND ANSWER

containerd manages runtime lifecycle and image-related tasks; runc is a low-level Linux container runtime; OCI defines interoperable image and runtime specifications.

↳ Draw this in the interview



WHY / HOW IT WORKS

Separating these responsibilities helps explain why an image built with Docker can be consumed by other compatible runtimes. It also helps locate failures: image retrieval, runtime creation and application startup are different steps.

COMMAND OR MINI EXAMPLE — from original guide

```
Application artifact -> OCI-compatible image
Engine/runtime stack -> container process
# The registry stores content, not a running runc process.
```

THE INTERVIEWER MAY ASK...

Does Kubernetes need Docker Engine to run a Docker-built image?

A. No. Kubernetes talks to a CRI-compatible runtime such as containerd or CRI-O; it can run OCI images built with Docker without using Docker Engine.

⚠ WATCH OUT — Product architecture can evolve. Do not assume a specific daemon chain is required by every OCI-compatible platform.

ORIGINAL QUESTION PRESERVED

Source guide p. 10

FOUNDATION

Q010 How does the writable container layer work?

THE 20-SECOND ANSWER

Filesystem changes go into the container's own layer above the image's read-only layers; mounted storage has separate behavior.

↳ Draw this in the interview



WHY / HOW IT WORKS

The change can be inspected after the process exits while the container object exists. Removing that object removes its writable layer. A later container from the same image has no note.txt unless it was in the image or a mount.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --name dk-q-edit alpine:3.22 \
  sh -c 'echo changed > /note.txt'
docker diff dk-q-edit
docker rm dk-q-edit
```

THE INTERVIEWER MAY ASK...

How does a named volume change this lifecycle?

A. A named volume has its own lifecycle. It keeps mounted data when the container is removed and can be attached to a replacement container.

⚠ WATCH OUT — This is not a persistence strategy for a database. A committed image also is not a consistent backup of a running database.

ORIGINAL QUESTION PRESERVED

Source guide p. 11

FOUNDATION

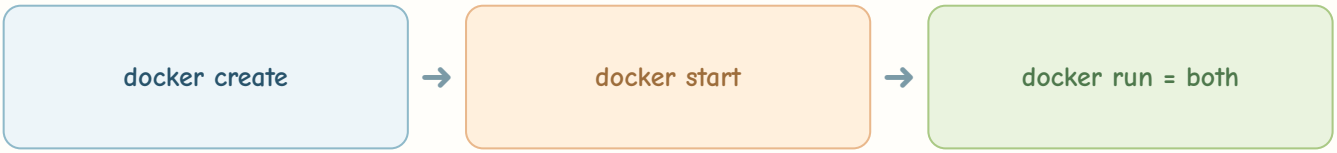
CHAPTER 03 — Container lifecycle and essential commands

Q011 What is the difference between run, create and start?

THE 20-SECOND ANSWER

run creates and starts a new container; create only creates it; start launches an existing stopped container.

↳ Draw this in the interview



WHY / HOW IT WORKS

The two start commands execute the existing container's configured command again. They do not create two new objects. In contrast, two unnamed run commands create two separate containers.

COMMAND OR MINI EXAMPLE — from original guide

```
docker create --name dk-q-once alpine:3.22 echo hello
docker start -a dk-q-once
docker start -a dk-q-once
docker rm dk-q-once
```

THE INTERVIEWER MAY ASK...

Can you change the startup environment with docker start?

A. No. `docker start` reuses the container's original environment and configuration. Create a replacement container with updated `-e` or Compose settings.

⚠ WATCH OUT — start does not rebuild an image or pick up a new image tag automatically. Recreate to change most container configuration.

ORIGINAL QUESTION PRESERVED

Source guide p. 12

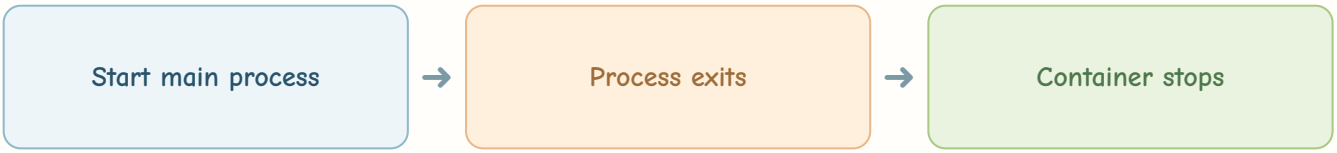
FOUNDATION

Q012 Why does a container exit immediately after starting?

THE 20-SECOND ANSWER

A container normally lives as long as its main process. A command that finishes, crashes or cannot execute makes the container exit.

↳ Draw this in the interview



WHY / HOW IT WORKS

echo prints once and exits successfully, so an exited container is expected. A missing executable or application exception requires a different diagnosis. Detached mode changes terminal attachment, not process lifetime.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --name dk-q-short alpine:3.22 echo done
docker ps -a --filter name=dk-q-short
docker logs dk-q-short
docker inspect dk-q-short --format '{{.State.ExitCode}}'
```

THE INTERVIEWER MAY ASK...

How would you distinguish a successful batch job from a failed API server?

A. Check exit code, logs, and the expected process type. Exit 0 is fine for a short batch task; an API that should stay up is failing its service contract.

⚠ WATCH OUT — Adding sleep forever hides startup failure and does not turn a broken service into a healthy one.

ORIGINAL QUESTION PRESERVED

Source guide p. 12

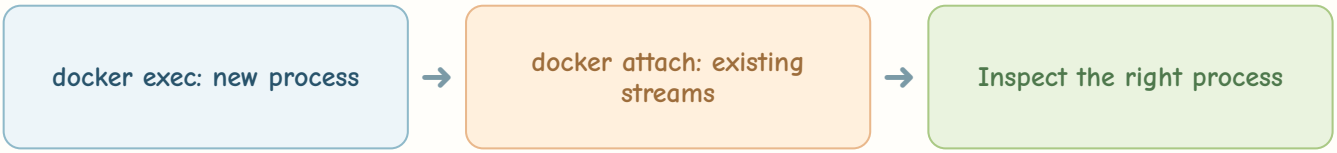
FOUNDATION

Q013 What is the difference between exec and attach?

THE 20-SECOND ANSWER

exec starts an additional process inside a running container; attach connects to the main process's standard streams.

↳ Draw this in the interview



WHY / HOW IT WORKS

nginx -t runs a separate configuration check. An exec shell can inspect files if the image contains a shell. Attach does not create a shell and interactive signals may affect the main process.

COMMAND OR MINI EXAMPLE — from original guide

```
docker exec dk-q-web nginx -t
docker exec -it dk-q-web sh
# attach targets the existing main process streams.
```

THE INTERVIEWER MAY ASK...

How would you inspect a crashed container when exec cannot run?

A. Use `docker logs`, `docker inspect`, `docker cp` on a retained container, and image metadata. Reproduce or debug with a separate disposable container if needed.

⚠ WATCH OUT — exec requires a running container and a present executable. Minimal images may provide neither sh nor bash.

ORIGINAL QUESTION PRESERVED

Source guide p. 13

FOUNDATION

Q014 How do stop, kill and rm differ?

THE 20-SECOND ANSWER

stop requests graceful termination before a timeout; kill sends a signal directly; rm removes the container object.

↳ Draw this in the interview



WHY / HOW IT WORKS

The default stop signal is SIGTERM unless configured otherwise; a surviving process is forcibly killed after the timeout. Removing the stopped container discards its writable layer, while its image remains available.

COMMAND OR MINI EXAMPLE — from original guide

```
docker stop --time 30 dk-q-web
docker inspect dk-q-web --format '{{.State.Status}}'
docker rm dk-q-web
```

THE INTERVIEWER MAY ASK...

How do you align Docker's timeout with a Spring shutdown timeout?

A. Set Docker's stop grace period longer than the Spring shutdown phases plus request drain and cleanup time. Verify SIGTERM reaches the JVM.

⚠ WATCH OUT — rm -f combines forced termination with removal. It gives the application no normal drain window and can erase useful diagnosis state.

ORIGINAL QUESTION PRESERVED

Source guide p. 13

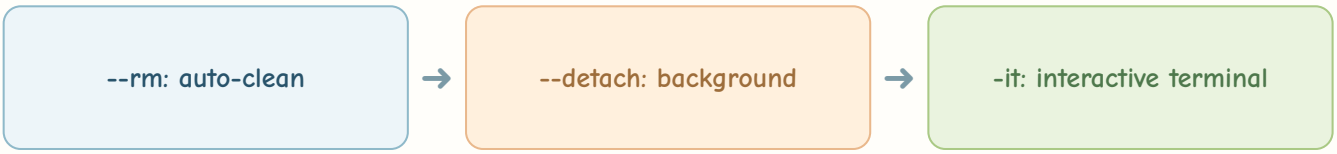
FOUNDATION

Q015 When should you use --rm, -d and -it?

THE 20-SECOND ANSWER

--rm removes an exiting container, -d detaches the terminal, and -it supplies interactive input plus a pseudo-terminal.

↳ Draw this in the interview



WHY / HOW IT WORKS

An ephemeral command is suitable for --rm; a server can run detached; a shell benefits from interactive terminal behavior. The flags solve different problems and can sometimes be combined.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --rm alpine:3.22 echo temporary
docker run -d --name dk-q-server nginx:stable
docker run --rm -it alpine:3.22 sh
```

THE INTERVIEWER MAY ASK...

Would you use --rm on a container whose crash you must investigate?

A. Usually no. Retain the exited container so `inspect`, filesystem differences, and logs are available; remove it after capturing incident evidence.

⚠ WATCH OUT — --rm does not remove explicitly named volumes. Automatic removal can also discard container metadata needed after a failure.

ORIGINAL QUESTION PRESERVED

Source guide p. 14

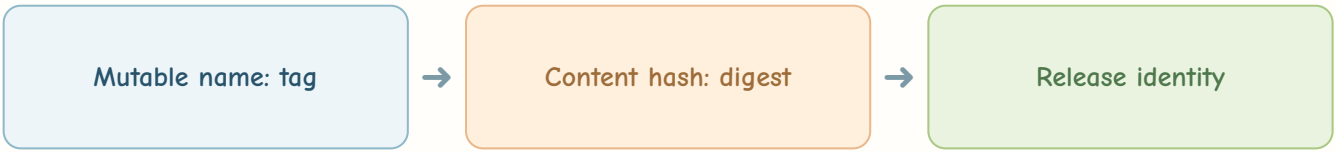
FOUNDATION

Q016 What is the difference between a tag and a digest?

THE 20-SECOND ANSWER

A tag is a mutable name; a digest identifies specific manifest content, including an image or multi-platform image index.

↳ Draw this in the interview



WHY / HOW IT WORKS

A registry owner can move stable to a new release. Recording a digest identifies the exact published content that was selected. An index digest can identify multiple platform variants; an individual manifest digest identifies one variant.

COMMAND OR MINI EXAMPLE — from original guide

```
docker pull nginx:stable
docker image inspect nginx:stable \
  --format '{{json .RepoDigests}}'
docker pull REGISTRY/TEAM/APP@sha256:DIGEST_HEX
```

THE INTERVIEWER MAY ASK...

What extra evidence would you require before deploying a pinned image?

A. Require trusted publisher/signature or provenance evidence, a reviewed SBOM and vulnerability findings, correct platform, license policy, and successful image tests.

⚠ WATCH OUT — A digest establishes content identity, not publisher trust, freshness or absence of vulnerabilities.

ORIGINAL QUESTION PRESERVED

Source guide p. 15

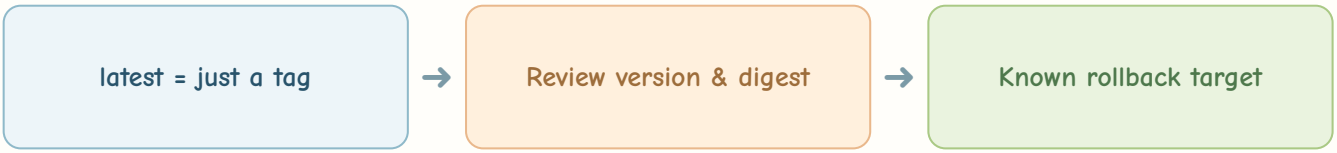
FOUNDATION

Q017 Does latest mean the newest or safest image?

THE 20-SECOND ANSWER

No. latest is a conventional tag whose meaning is chosen by the publisher; Docker does not compare all versions to calculate it.

↳ Draw this in the interview



WHY / HOW IT WORKS

An absent tag commonly selects latest, but that name may lag a release or point to an unsuitable build. A release tag improves readability; an enforced immutable tag or recorded digest improves reproducibility.

COMMAND OR MINI EXAMPLE — from original guide

```
docker pull TEAM/APP:latest
docker pull TEAM/APP:1.7.0
# Either tag can be moved if registry policy permits.
```

THE INTERVIEWER MAY ASK...

How would you identify a rollback target reliably?

A. Record the exact working image digest and release configuration in your deployment record. Roll back to that digest, not whatever a mutable tag means later.

⚠ WATCH OUT — A version-looking tag is also mutable unless the registry or workflow prevents reassignment.

ORIGINAL QUESTION PRESERVED

Source guide p. 15

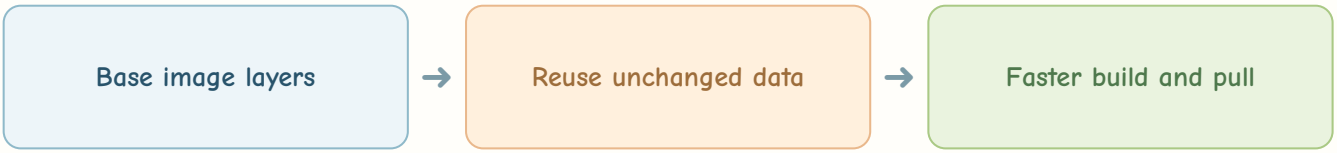
FOUNDATION

Q018 What are image layers, and why are they useful?

THE 20-SECOND ANSWER

Layers represent reusable filesystem changes; images reference those layers together with configuration metadata.

↳ Draw this in the interview



WHY / HOW IT WORKS

Shared layers reduce transfer and storage when builds reuse common content. A later deletion can hide a file in the resulting filesystem while its bytes remain in an earlier layer. Some instructions alter metadata rather than add filesystem content.

COMMAND OR MINI EXAMPLE — from original guide

```
docker image history IMAGE
docker image inspect IMAGE
# Two related images may reuse a common base layer.
```

THE INTERVIEWER MAY ASK…

Why should dependency downloads happen before frequently changing source code?

A. Stable dependency files are copied first so Docker can reuse the dependency layer when only source code changes, avoiding unnecessary package downloads.

⚠ WATCH OUT — history is not a complete audit of all file contents or of the supply chain. Never bake a credential into a layer and expect a later delete to erase it.

ORIGINAL QUESTION PRESERVED

Source guide p. 16

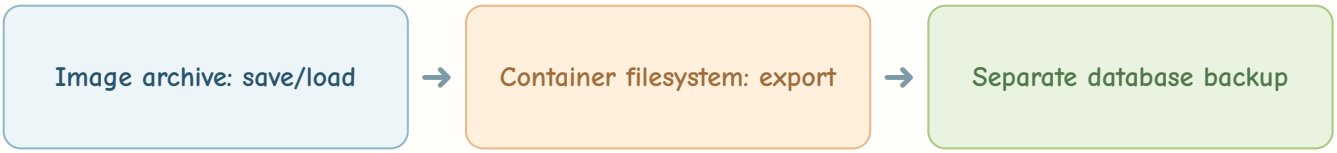
FOUNDATION

Q019 What is the difference between save/load and export/import?

THE 20-SECOND ANSWER

save/load transfers image data and tags; export/import operates on a container filesystem snapshot with different metadata and history behavior.

↳ Draw this in the interview



WHY / HOW IT WORKS

An image archive suits an offline image transfer. A container export is not equivalent to its original layered image and does not include mounted volume contents. Importing a filesystem also does not automatically restore the original runtime configuration.

COMMAND OR MINI EXAMPLE — from original guide

```
docker image save -o app-image.tar TEAM/APP:1.7.0
docker image load -i app-image.tar
# Registry push/pull is normally preferable for delivery.
```

THE INTERVIEWER MAY ASK...

How would you move a service image and its database data separately?

A. Move the image using a registry or `docker save/load`. Back up and restore database data with database-aware tools and separately verify integrity.

⚠ WATCH OUT — Neither path is a substitute for application-consistent database backup. Protect archives that may contain private code or configuration.

ORIGINAL QUESTION PRESERVED

Source guide p. 16

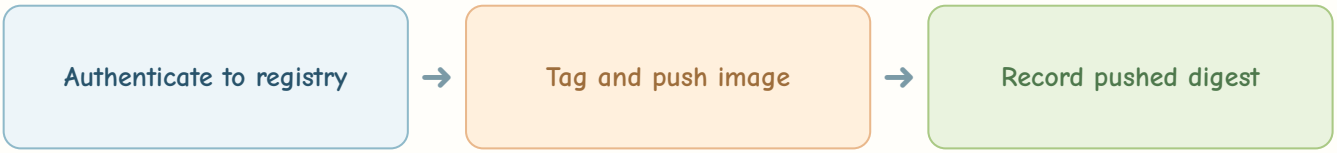
FOUNDATION

Q020 How do you push a local image to a private registry?

THE 20-SECOND ANSWER

Authenticate to the registry, tag the image under its repository name, push it, and record the resulting content identity.

↳ Draw this in the interview



WHY / HOW IT WORKS

The registry hostname and repository path identify the destination. Authentication and repository write permission must both be valid. CI should use a protected credential mechanism and capture the digest produced by publishing.

COMMAND OR MINI EXAMPLE — from original guide

```
docker login registry.example.com
docker tag dk-app:local registry.example.com/team/app:1.7.0
docker push registry.example.com/team/app:1.7.0
```

THE INTERVIEWER MAY ASK…

Why should a CI release deploy the pushed digest rather than rebuild from source?

A. A second build might resolve changed base tags, dependencies, or build inputs. Promoting the tested digest deploys exactly the reviewed artifact.

⚠ WATCH OUT — A local image name does not create a remote repository or grant access. Avoid embedding credentials in command history or image build arguments.

ORIGINAL QUESTION PRESERVED

Source guide p. 17

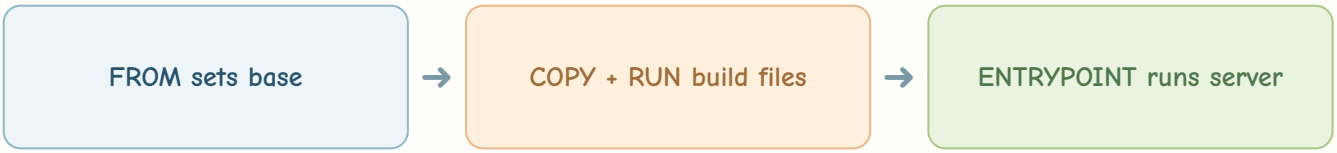
BUILD / RUNTIME

Q021 What are FROM, WORKDIR, COPY and RUN used for?

THE 20-SECOND ANSWER

FROM selects a stage base, WORKDIR sets a path, COPY adds files from a source, and RUN executes a build-time command.

↳ Draw this in the interview



WHY / HOW IT WORKS

The Dockerfile assumes target/app.jar exists in the build context. The test runs during image construction, not whenever a container starts. WORKDIR also creates a missing directory as part of its defined behavior.

COMMAND OR MINI EXAMPLE — from original guide

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/app.jar /app/app.jar
RUN test -r /app/app.jar
```

THE INTERVIEWER MAY ASK...

Which instruction should launch the server at container startup?

A. Use exec-form `ENTRYPOINT` for the main server command; use `CMD` for optional default arguments. `RUN` is for image build time.

⚠ WATCH OUT — RUN java -jar app.jar starts the server during the build and is usually the wrong lifecycle for a web application.

ORIGINAL QUESTION PRESERVED

Source guide p. 18

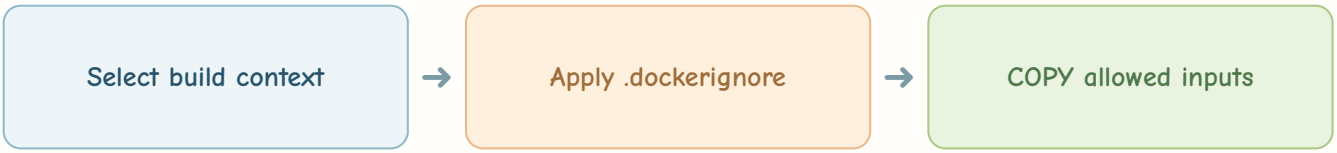
BUILD / RUNTIME

Q022 What is the build context, and what does .dockerignore do?

THE 20-SECOND ANSWER

The context is the set of build inputs made available to the builder; .dockerignore removes matching inputs from that context.

↳ Draw this in the interview



WHY / HOW IT WORKS

A source-build context can exclude old artifacts and private local files. A prebuilt-JAR workflow needs a different ignore policy. COPY cannot simply reach an arbitrary parent directory outside the selected context.

COMMAND OR MINI EXAMPLE — from original guide

```
# .dockerignore
.git
.env
secrets/
target/
# For a Dockerfile that copies a prebuilt JAR, do NOT ignore target/.
```

THE INTERVIEWER MAY ASK…

Why can a working local build fail when CI uses a smaller context?

A. CI may exclude files that a broad local context supplied accidentally. Compare context paths, `.dockerignore`, COPY sources, and required build artifacts.

⚠ WATCH OUT — .gitignore and .dockerignore serve different tools. A .gitignored secret can still enter the Docker context unless excluded there too.

ORIGINAL QUESTION PRESERVED

Source guide p. 18

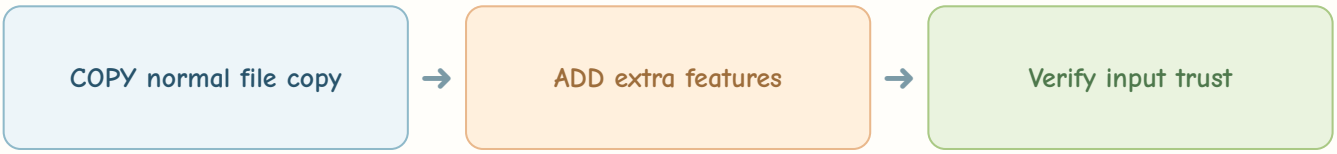
BUILD / RUNTIME

Q023 How are COPY and ADD different?

THE 20-SECOND ANSWER

COPY performs explicit copying; ADD also supports capabilities such as fetching supported remote sources and extracting local tar archives.

↳ Draw this in the interview



WHY / HOW IT WORKS

Explicit copying makes application packaging easier to audit. If remote retrieval is required, account for source identity, validation and cache behavior rather than hiding that decision in a generic instruction.

COMMAND OR MINI EXAMPLE — from original guide

```
COPY app.jar /app/app.jar
COPY config/ /app/config/
# Choose ADD only when its extra source behavior is intentional.
```

THE INTERVIEWER MAY ASK…

How would you verify the bytes of a downloaded dependency?

A. Pin the expected SHA-256 hash or signed release and verify it during the build. Prefer trusted artifact repositories rather than an unchecked URL.

⚠ WATCH OUT — Do not assume a remote archive is extracted like a local tar archive. Features and flags can depend on the Dockerfile frontend version.

ORIGINAL QUESTION PRESERVED

Source guide p. 19

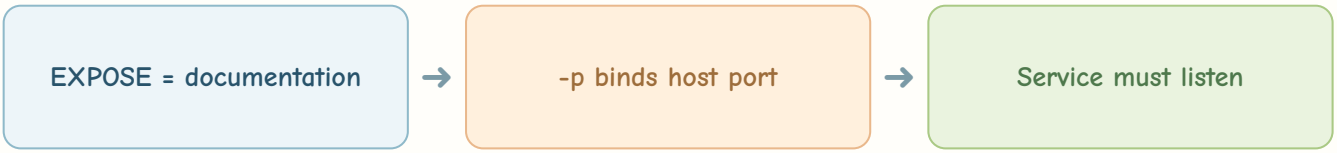
BUILD / RUNTIME

Q024 Does EXPOSE make a port reachable from the host?

THE 20-SECOND ANSWER

No. EXPOSE documents intended container ports; runtime publication or another network path provides reachability.

↳ Draw this in the interview



WHY / HOW IT WORKS

The application must actually listen on port 8080 inside the container. The runtime mapping directs host port 18080 to it. Same-network containers usually connect directly to the container port without a host mapping.

COMMAND OR MINI EXAMPLE — from original guide

```
# Dockerfile
EXPOSE 8080
# Runtime on a local development machine
docker run -p 127.0.0.1:18080:8080 TEAM/APP:1.7.0
```

THE INTERVIEWER MAY ASK...

Can two containers expose 8080 while one host cannot bind 18080 twice?

A. Yes. Each container has its own port 8080. Host port 18080 is one binding, so use different host ports or a reverse proxy.

⚠ WATCH OUT — Writing EXPOSE 8080 does not change a Spring server configured to listen on 9090.

ORIGINAL QUESTION PRESERVED

Source guide p. 19

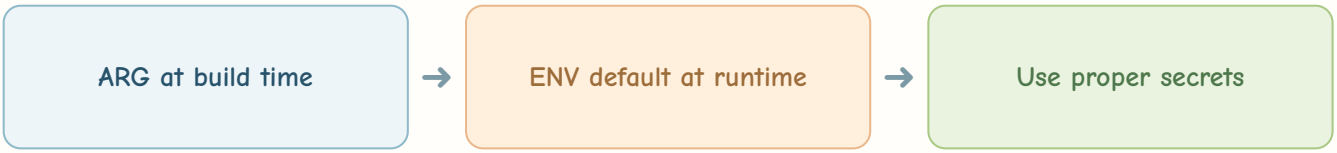
BUILD / RUNTIME

Q025 What is the difference between ARG and ENV?

THE 20-SECOND ANSWER

ARG is a build-time variable; ENV establishes image environment defaults that runtime configuration can override.

↳ Draw this in the interview



WHY / HOW IT WORKS

APP_VERSION helps label the build; SERVER_PORT is available to the application at startup. ARG scope matters across stages and before FROM. Neither instruction is designed to safely hide credentials.

COMMAND OR MINI EXAMPLE — from original guide

```
ARG APP_VERSION=dev
LABEL org.opencontainers.image.version=$APP_VERSION
ENV SERVER_PORT=8080
# Runtime override:
docker run -e SERVER_PORT=9090 TEAM/APP:1.7.0
```

THE INTERVIEWER MAY ASK...

Should a database password be a build input or a runtime dependency?

A. It is a runtime dependency: inject it through an approved secret provider or mounted secret at startup. Never bake production passwords into images or build args.

⚠ WATCH OUT — Build arguments and environment values can surface in image metadata, history, provenance or process configuration. Use dedicated secret mechanisms.

ORIGINAL QUESTION PRESERVED

Source guide p. 20

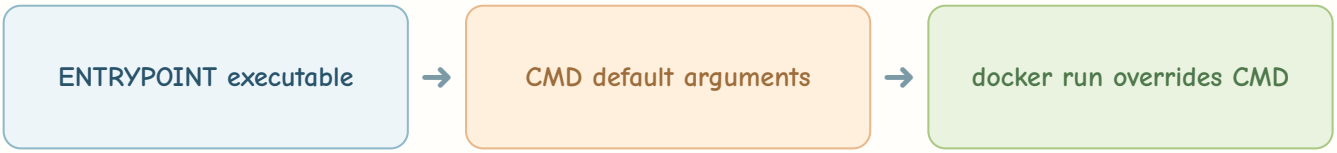
BUILD / RUNTIME

Q026 How do ENTRYPOINT and CMD work together?

THE 20-SECOND ANSWER

ENTRYPOINT supplies the executable; CMD supplies a default command or default arguments depending on the chosen configuration.

↳ Draw this in the interview



WHY / HOW IT WORKS

Arguments after the image replace CMD but keep the exec-form ENTRYPOINT. The application receives the supplied Spring argument. This makes a fixed executable configurable without a shell wrapper.

COMMAND OR MINI EXAMPLE — from original guide

```
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
CMD ["--server.port=8080"]
# docker run IMAGE --server.port=9090
# Starts: java -jar /app/app.jar --server.port=9090
```

THE INTERVIEWER MAY ASK…

What does --entrypoint change, and when is it useful?

A. `--entrypoint` replaces the image's default executable. Use it for an authorized one-off diagnostic or alternate program, taking care with arguments and privileges.

⚠ WATCH OUT — If an image has only CMD, arguments after the image can replace the entire command. Always inspect the image's actual defaults.

ORIGINAL QUESTION PRESERVED

Source guide p. 21

BUILD / RUNTIME

Q027 Why is exec form often preferred for a server?

THE 20-SECOND ANSWER

Exec form avoids an implicit shell and makes the configured executable directly receive container-level signals in the normal setup.

↳ Draw this in the interview



WHY / HOW IT WORKS

Direct Java startup removes a common reason that SIGTERM never reaches the JVM. Shell form can have different argument and signal behavior, especially when a wrapper launches children without replacing itself.

COMMAND OR MINI EXAMPLE — from original guide

```
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
# Shell form adds a shell layer:
# ENTRYPOINT java -jar /app/app.jar
```

THE INTERVIEWER MAY ASK…

Why can a container stop take exactly its forced-kill timeout?

A. The main process may ignore SIGTERM, a wrapper may not forward it, or shutdown may exceed the grace period. Docker then sends SIGKILL at the deadline.

⚠ WATCH OUT — Exec form does not perform shell expansion: ["echo", "$NAME"] prints a literal argument. Use application configuration or a deliberate wrapper when needed.

ORIGINAL QUESTION PRESERVED

Source guide p. 22

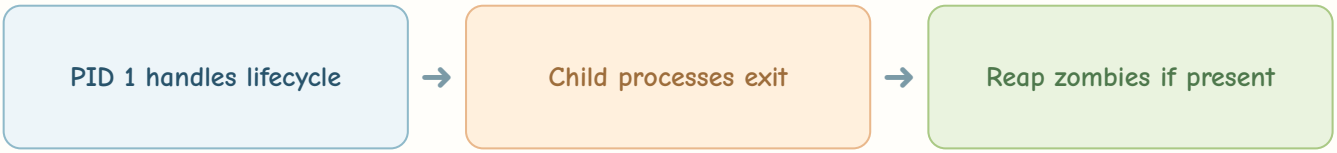
BUILD / RUNTIME

Q028 What is special about PID 1 in a Linux container?

THE 20-SECOND ANSWER

The main process occupies the container's initial process role and must handle termination and any child-process lifecycle responsibilities it creates.

↳ Draw this in the interview



WHY / HOW IT WORKS

exec replaces the wrapper so Java becomes the main application process. --init inserts a small init process that can forward signals and reap orphaned children. These solve distinct process-management concerns.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --init IMAGE
# If a wrapper is needed:
#!/bin/sh
set -eu
exec java -jar /app/app.jar "$@"
```

THE INTERVIEWER MAY ASK...

How would you detect accumulating zombie child processes?

A. Inspect process states with `docker top` or host process tools; zombie children usually appear with state `Z`. Check whether PID 1 reaps exited children.

⚠ WATCH OUT — A Java server that does not spawn unmanaged children may not require an extra init. Adding one does not repair an application that ignores shutdown.

ORIGINAL QUESTION PRESERVED

Source guide p. 22

BUILD / RUNTIME

Q029 How do you pass JVM options without breaking arguments?

THE 20-SECOND ANSWER

Use Java's supported option environment variables or a wrapper with deliberate argument handling; do not expect shell expansion inside an exec array.

↳ Draw this in the interview



WHY / HOW IT WORKS

The JVM consumes JAVA_TOOL_OPTIONS when it initializes; the application still receives normal command arguments. The literal $JAVA_OPTS in an exec array would not be expanded into JVM flags.

COMMAND OR MINI EXAMPLE — from original guide

```
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
docker run -e JAVA_TOOL_OPTIONS='-Xms128m -Xmx384m' IMAGE
# Do not write ["java", "$JAVA_OPTS", "-jar", "app.jar"].
```

THE INTERVIEWER MAY ASK...

How would you verify the heap size used inside the running container?

A. Inspect `jcmd <pid> VM.flags` or `java -XX:+PrintFlagsFinal -version` inside a suitable image, plus container cgroup values and runtime telemetry.

⚠ WATCH OUT — Supported option variables have specific parsing and applicability. Avoid secrets in JVM flags and confirm effective settings on the exact JDK.

ORIGINAL QUESTION PRESERVED

Source guide p. 23

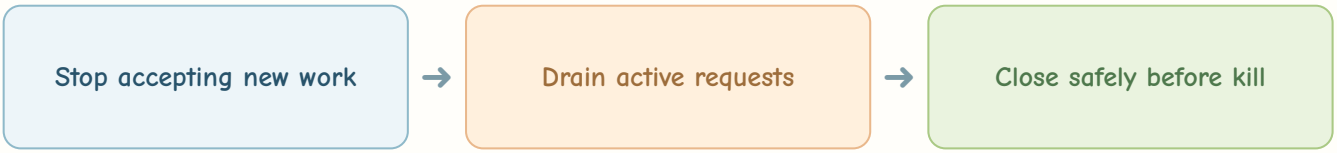
BUILD / RUNTIME

Q030 Why should a Spring Boot container stop gracefully?

THE 20-SECOND ANSWER

Graceful shutdown drains accepted work and closes resources before the runtime forcibly ends the process.

↳ Draw this in the interview



WHY / HOW IT WORKS

The JVM must receive the stop signal, and the platform must withdraw traffic while the application drains. Spring's timeout is per shutdown phase; total shutdown and load-balancer propagation can need additional time.

COMMAND OR MINI EXAMPLE — from original guide

```
# application.properties - supported Spring Boot baseline
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=20s
# Docker gives the process a larger total budget:
docker stop --time 45 APP_CONTAINER
```

THE INTERVIEWER MAY ASK...

What happens to an in-flight database transaction when the process is finally killed?

A. If the process is killed before commit, the database ordinarily rolls back the open transaction when its connection closes. External effects may still need reconciliation.

⚠ WATCH OUT — Graceful HTTP shutdown does not automatically make Kafka processing, scheduled jobs or external side effects exactly once.

ORIGINAL QUESTION PRESERVED

Source guide p. 23

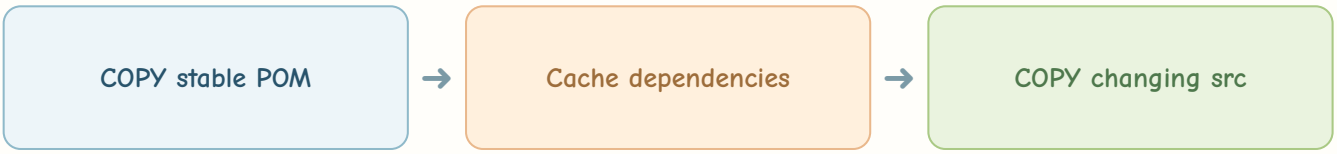
BUILD / RUNTIME

Q031 How does Docker build caching work?

THE 20-SECOND ANSWER

The builder reuses results when relevant instruction inputs match its cache; a changed dependency can invalidate later dependent steps.

↳ Draw this in the interview



WHY / HOW IT WORKS

Changing only source can reuse the earlier dependency step, while changing pom.xml invalidates it. A cache mount preserves downloaded artifacts between builds without copying that cache into the final image by itself.

COMMAND OR MINI EXAMPLE — from original guide

```
COPY pom.xml ./
RUN --mount=type=cache,target=/root/.m2 \
    mvn -B dependency:go-offline
COPY src/ src/
RUN --mount=type=cache,target=/root/.m2 mvn -B package
```

THE INTERVIEWER MAY ASK...

How would you measure whether this change actually speeds up CI?

A. Record cold and warm CI build durations and cache-hit output over repeated representative changes. Compare medians, total pipeline time, and correctness.

⚠ WATCH OUT — dependency:go-offline does not guarantee every project will later build without network access. Maven extensions and profiles can introduce extra inputs.

ORIGINAL QUESTION PRESERVED

Source guide p. 24

BUILD / RUNTIME

Q032 Why is COPY . . near the start of a Dockerfile often inefficient?

THE 20-SECOND ANSWER

It makes unrelated context changes inputs to expensive downstream build steps.

↳ Draw this in the interview



WHY / HOW IT WORKS

A changed README can cause a rebuild if it is included in the broad copy. Copying the required build descriptors first and source later narrows invalidation, provided all required project files are included.

COMMAND OR MINI EXAMPLE — from original guide

```
# Less selective:
COPY . .
RUN mvn -B package
# Better structure separates stable build files from changing source.
```

THE INTERVIEWER MAY ASK…

Which files must a multi-module Maven dependency step receive?

A. Copy the root POM, module POMs, parent descriptors, Maven wrapper/config, and any version catalogs or files used for dependency resolution.

⚠ WATCH OUT — Incorrectly excluding generated sources or parent POMs makes the optimization fail. Correct inputs matter more than a high cache-hit percentage.

ORIGINAL QUESTION PRESERVED

Source guide p. 25

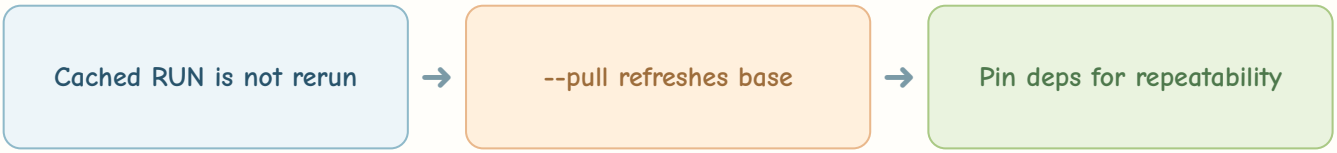
BUILD / RUNTIME

Q033 Does a cached RUN automatically refresh remote package versions?

THE 20-SECOND ANSWER

No. Reusing a cached result does not rerun its network requests just because a repository changed.

↳ Draw this in the interview



WHY / HOW IT WORKS

--pull checks referenced image sources for updates. --no-cache avoids reusing ordinary build-step results. Neither alone pins application dependencies or makes two builds bit-for-bit reproducible.

COMMAND OR MINI EXAMPLE — from original guide

```
docker build --pull -t dk-app:fresh-base .
docker build --no-cache --pull -t dk-app:rebuild .
# --pull and --no-cache address different sources of reuse.
```

THE INTERVIEWER MAY ASK...

How do you combine update automation with reproducible release builds?

A. Use automated dependency/base updates that create reviewed changes; pin dependencies and digests for release builds, then test and promote one exact image.

⚠ WATCH OUT — Persistent cache mounts may still contain downloaded artifacts. Floating package versions can change during a fresh build.

ORIGINAL QUESTION PRESERVED

Source guide p. 25

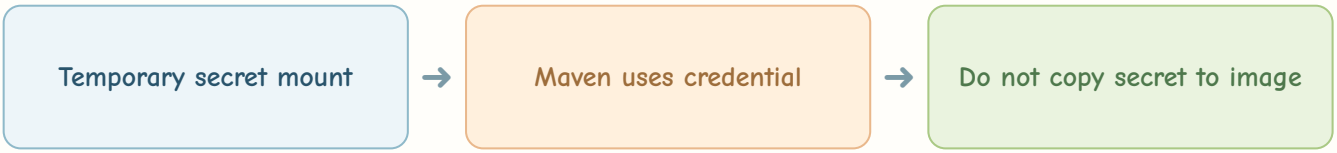
BUILD / RUNTIME

Q034 How do build secrets differ from copying a secret file?

THE 20-SECOND ANSWER

A BuildKit secret mount makes a secret available to a selected build step without deliberately adding it as ordinary image content.

↳ Draw this in the interview



WHY / HOW IT WORKS

The build step reads Maven credentials from its temporary secret mount. The final image should contain the application artifact, not settings.xml. The command must still avoid writing the secret into logs or generated files.

COMMAND OR MINI EXAMPLE — from original guide

```
RUN --mount=type=secret,id=maven_settings,required=true \
    mvn -B -s /run/secrets/maven_settings package
docker build --secret id=maven_settings,src=settings.xml .
```

THE INTERVIEWER MAY ASK…

How would you force a credential-sensitive build step to rerun safely?

A. Change a non-secret cache-busting version argument or use `--no-cache` for the relevant rebuild. Never put the secret content in an ARG or build log.

⚠ WATCH OUT — Secret rotation alone does not invalidate a cached step. Secret mounts reduce exposure but do not make a malicious build trustworthy.

ORIGINAL QUESTION PRESERVED

Source guide p. 26

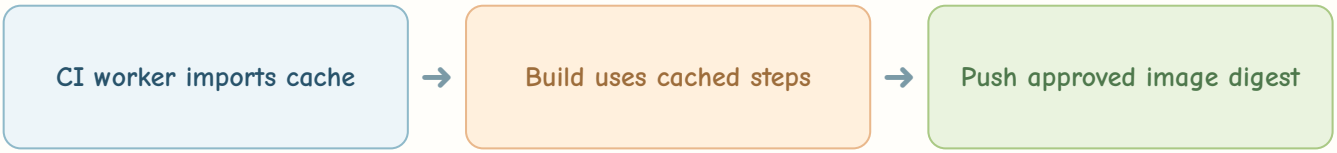
BUILD / RUNTIME

Q035 How can ephemeral CI workers share build cache?

THE 20-SECOND ANSWER

Import and export cache through a supported external backend, while keeping cache identity and permissions separate from release identity.

↳ Draw this in the interview



WHY / HOW IT WORKS

A new worker can reuse prior results instead of rebuilding everything. The cache reference is an optimization artifact; the published application digest remains the release identity. Backend support depends on builder configuration.

COMMAND OR MINI EXAMPLE — from original guide

```
docker buildx build \
  --cache-from type=registry,ref=REGISTRY/APP:buildcache \
  --cache-to type=registry,ref=REGISTRY/APP:buildcache,mode=max \
  --tag REGISTRY/APP:BUILD_ID --push .
```

THE INTERVIEWER MAY ASK…

What prevents two concurrent jobs from replacing each other's cache?

A. Use cache keys unique to branch/build, immutable keys where possible, and restricted write permissions. Promote a selected cache only with controlled updates.

⚠ WATCH OUT — Do not grant untrusted pull-request builds permission to overwrite trusted caches or release tags. Cache misses must not break correctness.

ORIGINAL QUESTION PRESERVED

Source guide p. 26

BUILD / RUNTIME

Q036 What is a multi-stage build?

THE 20-SECOND ANSWER

It uses separate build stages and selectively transfers outputs into a final runtime stage.

↳ Draw this in the interview



WHY / HOW IT WORKS

Maven and source files stay in the build stage unless explicitly copied to the final stage. This example requires the Maven project to produce target/app.jar and run appropriate tests.

COMMAND OR MINI EXAMPLE — from original guide

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
COPY src/ src/
RUN mvn -B package
FROM eclipse-temurin:21-jre
COPY --from=build /src/target/app.jar /app/app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

THE INTERVIEWER MAY ASK...

Which artifacts would you keep for debugging without shipping them in the runtime image?

A. Keep test reports, SBOMs, JAR checksums, symbol/debug artifacts, and build logs in protected CI storage. Ship only the necessary runtime outputs.

⚠ WATCH OUT — Multi-stage packaging does not prove that the JAR is valid or that unused secrets never entered intermediate build output.

ORIGINAL QUESTION PRESERVED

Source guide p. 27

BUILD / RUNTIME

Q037 Should a Java runtime image use a JDK, JRE, distroless base or jlink?

THE 20-SECOND ANSWER

Choose the smallest maintained runtime that satisfies application modules, native dependencies, observability and operational needs.

↳ Draw this in the interview



WHY / HOW IT WORKS

A JRE-based server can run an executable Spring JAR without a compiler. A stripped runtime may need explicit modules, CA certificates or native libraries. Reduced package count can help, but support and diagnosis remain design requirements.

COMMAND OR MINI EXAMPLE — from original guide

```
JDK image:        compiler and diagnostic tools available
JRE image:        runtime-focused packaging
Custom jlink:     selected modules and explicit maintenance
Distroless:       reduced OS tools; inspect by other means
```

THE INTERVIEWER MAY ASK…

How would you investigate a JVM issue when jcmd is absent?

A. Collect JVM metrics, JFR or thread dumps via supported attach/debug tools or a separate debug image; temporarily reproduce with a diagnostic JDK image.

⚠ WATCH OUT — A shell-free image cannot run a shell-based health check. jlink does not automatically discover every dynamically loaded dependency.

ORIGINAL QUESTION PRESERVED

Source guide p. 28

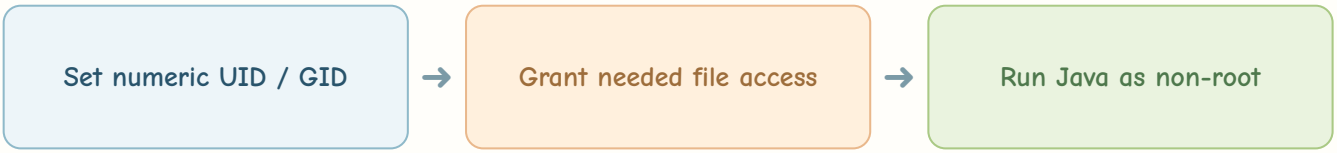
BUILD / RUNTIME

Q038 How do you run a Java container as a non-root user?

THE 20-SECOND ANSWER

Set a runtime UID and GID, make application files readable, and provide writable directories only where required.

↳ Draw this in the interview



WHY / HOW IT WORKS

Linux can run this process with a numeric identity even without a named account. The application needs readable code and a writable temp location. Mounted files are governed by their actual ownership and mount permissions.

COMMAND OR MINI EXAMPLE — from original guide

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --chown=10001:10001 target/app.jar /app/app.jar
USER 10001:10001
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

THE INTERVIEWER MAY ASK...

How would you fix a permission error without making the whole application root?

A. Check numeric UID/GID and actual mount permissions. Grant only the necessary directory access using ownership, group IDs, or a dedicated writable mount.

⚠ WATCH OUT — COPY --chown changes copied image files, not arbitrary host bind mounts. Applications that look up a named user may need an account entry.

ORIGINAL QUESTION PRESERVED

Source guide p. 28

BUILD / RUNTIME

Q039 Why avoid COPY target/*.jar app.jar without checking the outputs?

THE 20-SECOND ANSWER

Builds can produce multiple JARs, and a wildcard can select unintended artifacts or fail when several sources target a single filename.

↳ Draw this in the interview



WHY / HOW IT WORKS

An explicit output name makes the Dockerfile contract clear. Spring repackage plugins and project conventions can produce auxiliary files, so inspect actual build output rather than guessing from a wildcard.

COMMAND OR MINI EXAMPLE — from original guide

```
# pom.xml build configuration
<build>
  <finalName>app</finalName>
</build>
# Dockerfile
COPY target/app.jar /app/app.jar
```

THE INTERVIEWER MAY ASK...

How would CI verify that the copied JAR came from the tested build?

A. CI should test the JAR, compute its SHA-256, and copy that same named artifact into the image. Check the hash or build attestation after packaging.

⚠ WATCH OUT — The example is a POM fragment, not a complete Maven project. Multi-module builds need the correct module artifact path.

ORIGINAL QUESTION PRESERVED

Source guide p. 29

BUILD / RUNTIME

Q040 How are Spring Boot buildpacks different from a hand-written Dockerfile?

THE 20-SECOND ANSWER

Buildpacks automate supported application detection and runtime image assembly; a Dockerfile gives direct control over packaging instructions.

↳ Draw this in the interview



WHY / HOW IT WORKS

Buildpacks can standardize Java runtime selection and image layers across teams. A Dockerfile can handle specialized native libraries or startup contracts explicitly. Both approaches need maintained bases and a reproducible delivery policy.

COMMAND OR MINI EXAMPLE — from original guide

```
./mvnw spring-boot:build-image \
  -Dspring-boot.build-image.imageName=dk-app:local
# Dockerfile alternative:
docker build -t dk-app:local .
```

THE INTERVIEWER MAY ASK...

Which approach would you standardize for twenty Spring services, and why?

A. For standard Spring services, use approved buildpacks when they fit: simpler shared updates. Use a reviewed Dockerfile for special native or startup needs.

⚠ WATCH OUT — Plugin version, builder and run image affect the result. Neither approach removes the need to test the produced image and record its digest.

ORIGINAL QUESTION PRESERVED

Source guide p. 29

BUILD / RUNTIME

Q041 What is the difference between a volume and a bind mount?

THE 20-SECOND ANSWER

A volume is managed by Docker; a bind mount exposes a specific path from the engine host inside the container.

↳ Draw this in the interview



WHY / HOW IT WORKS

The named volume outlives the temporary writer container. A bind mount connects directly to an existing host path, useful for development or managed config files. The storage location belongs to the engine host.

COMMAND OR MINI EXAMPLE — from original guide

```
docker volume create dk-q-data
docker run --rm --mount type=volume,src=dk-q-data,dst=/data \
  alpine:3.22 sh -c 'echo hello > /data/message'
# Bind example: --mount type=bind,src=HOST_PATH,dst=/config,readonly
```

THE INTERVIEWER MAY ASK...

Which mount would you choose for a disposable source checkout versus database data?

A. Use a bind mount for disposable development source files; choose a managed named volume or dedicated persistent storage for database files, plus backups.

⚠ WATCH OUT — Remote engines and Desktop file sharing change how host paths are reached. A bind mount can allow modification of host files unless restricted.

ORIGINAL QUESTION PRESERVED

Source guide p. 30

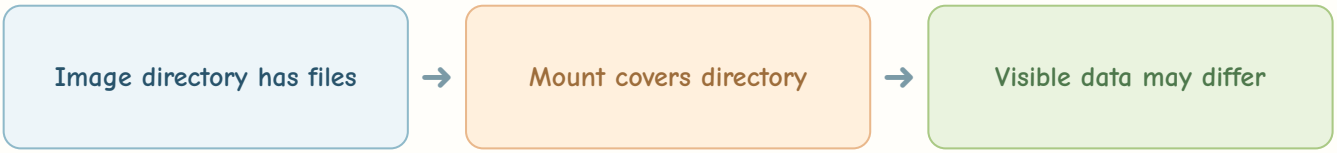
BUILD / RUNTIME

Q042 What happens when a mount targets a non-empty image directory?

THE 20-SECOND ANSWER

The mount obscures the image content at that path; empty named volumes also have defined initial copy behavior unless disabled.

↳ Draw this in the interview



WHY / HOW IT WORKS

A missing configuration file can be caused by the mount overlay rather than a broken COPY. Docker can populate a fresh empty volume from existing container-path content; a bind mount does not perform that volume initialization.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --rm \
  --mount type=bind,src=HOST_CONFIG,dst=/app/config,readonly \
  IMAGE
# Files from /app/config in the image are hidden by this mount.
```

THE INTERVIEWER MAY ASK...

Why might two deployments see different initial directory contents?

A. One deployment might mount a fresh volume that gets initial image content copied in; another may mount an existing volume or host bind that hides the image files.

⚠ WATCH OUT — Do not rely on volume initialization as an application migration system. Existing data and volume-nocopy alter the outcome.

ORIGINAL QUESTION PRESERVED

Source guide p. 31

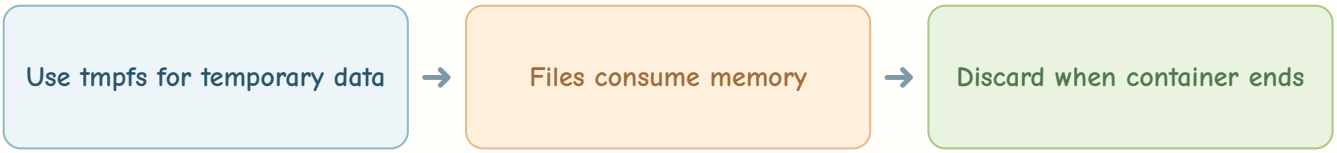
BUILD / RUNTIME

Q043 What does tmpfs provide, and when is it appropriate?

THE 20-SECOND ANSWER

A Linux tmpfs mount supplies temporary memory-backed filesystem storage outside the container's writable image layer.

↳ Draw this in the interview



WHY / HOW IT WORKS

The root filesystem is read-only but /tmp accepts temporary files. This suits scratch data that need not survive a stop. Its capacity still contributes to host and applicable container memory pressure.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --rm --read-only \
  --tmpfs /tmp:rw,size=64m \
  alpine:3.22 sh -c 'echo scratch > /tmp/note'
```

THE INTERVIEWER MAY ASK...

How would a Java application that creates large temporary files behave with this limit?

A. Large temp files consume tmpfs memory and can hit the tmpfs or container memory limit. Prefer bounded disk-backed temporary storage when files can be large.

⚠ WATCH OUT — tmpfs is not durable storage or a guarantee that data never reaches swap. Availability and behavior are platform-dependent.

ORIGINAL QUESTION PRESERVED

Source guide p. 31

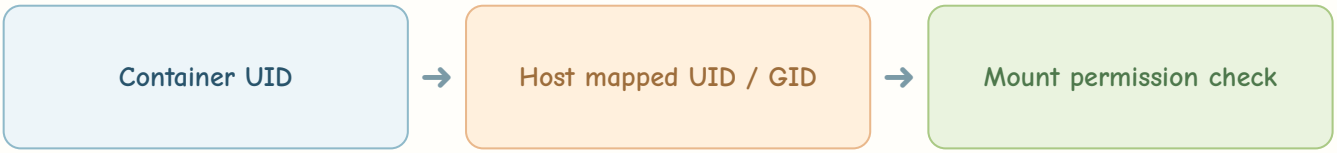
BUILD / RUNTIME

Q044 Why can a non-root container fail to write to a mounted directory?

THE 20-SECOND ANSWER

The process UID/GID, mounted file ownership, permissions and security labels must permit the operation.

↳ Draw this in the interview



WHY / HOW IT WORKS

An image directory owned by UID 10001 does not make a replacement bind mount owned by that UID. On Linux, user namespace mappings and SELinux policy may add further considerations.

COMMAND OR MINI EXAMPLE — from original guide

```
docker inspect APP_CONTAINER --format '{{json .Mounts}}'
docker exec APP_CONTAINER id
# On the engine host, inspect the specific mount's ownership.
# Provide the needed directory permissions for UID 10001.
```

THE INTERVIEWER MAY ASK...

How do user namespace mappings change the host UID you should inspect?

A. Container UIDs may map to different host UIDs under user-namespace remapping. Inspect `/proc/*/uid_map` and effective ownership at the host mount path.

⚠ WATCH OUT — chmod 777 or switching to root can mask the underlying mismatch. Avoid recursively changing ownership of an unrelated host tree.

ORIGINAL QUESTION PRESERVED

Source guide p. 32

BUILD / RUNTIME

Q045 Does removing a container delete its named volume?

THE 20-SECOND ANSWER

Normally no. Named volumes have a separate lifecycle and require explicit removal when no longer needed.

↳ Draw this in the interview



WHY / HOW IT WORKS

A replacement container can mount the retained volume. That separation enables persistence, but also leaves data behind after a service is removed. Anonymous volume cleanup and Compose down -v have additional rules.

COMMAND OR MINI EXAMPLE — from original guide

```
docker rm STOPPED_CONTAINER
docker volume inspect NAMED_VOLUME
# Only for disposable, identified data:
docker volume rm NAMED_VOLUME
```

THE INTERVIEWER MAY ASK...

How would you prove that restoring a backup recreates the required application state?

A. Restore the backup into a disposable clean environment, start the service, and verify data counts, application invariants, permissions, and a known test transaction.

⚠ WATCH OUT — Broad prune commands and down -v can remove valuable data. Volume persistence does not imply backup, replication or disaster recovery.

ORIGINAL QUESTION PRESERVED

Source guide p. 32

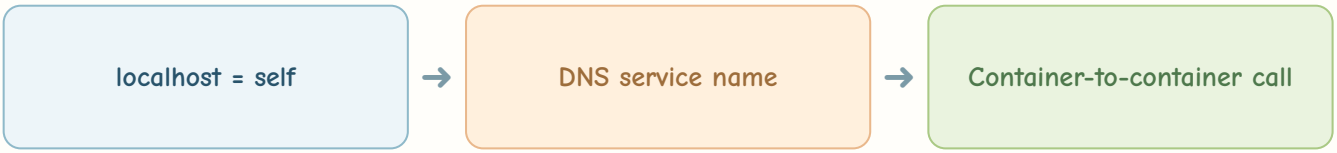
BUILD / RUNTIME

Q046 Why does localhost fail when one container calls another?

THE 20-SECOND ANSWER

localhost refers to the calling container's own network namespace in the normal isolated setup.

↳ Draw this in the interview



WHY / HOW IT WORKS

The app connects to the service DNS name db and its internal port. It does not use the host's published database port. A host-side test client instead uses a published endpoint appropriate to the host.

COMMAND OR MINI EXAMPLE — from original guide

```
# app and db share a user-defined network:
SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/orders
# Incorrect for a separate db container:
# jdbc:postgresql://localhost:5432/orders
```

THE INTERVIEWER MAY ASK...

What address should a third container use to call the app?

A. Use the service name and container port, such as `http://api:8080`, when both containers share a user-defined network. Do not use localhost.

⚠ WATCH OUT — Sharing a network does not guarantee the database is accepting connections or that authentication succeeds.

ORIGINAL QUESTION PRESERVED

Source guide p. 33

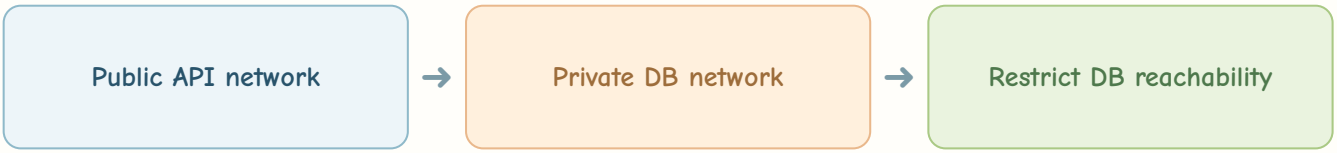
BUILD / RUNTIME

Q047 What is the difference between the default bridge and a user-defined bridge?

THE 20-SECOND ANSWER

User-defined bridges provide automatic name-based discovery for connected containers and a configurable grouping boundary.

↳ Draw this in the interview



WHY / HOW IT WORKS

The client resolves dk-q-web through Docker networking and reaches port 80 directly. Default bridge containers do not receive the same automatic container-name discovery behavior. Compose usually creates a project network for services.

COMMAND OR MINI EXAMPLE — from original guide

```
docker network create dk-q-net
docker run -d --name dk-q-web --network dk-q-net nginx:stable
docker run --rm --network dk-q-net alpine:3.22 \
  wget -qO- http://dk-q-web
```

THE INTERVIEWER MAY ASK...

Why would you attach an API to two networks but keep a database on only one?

A. Put the API on both public-facing and private database networks. Keep the database only on the private network to reduce direct reachability.

⚠ WATCH OUT — Network membership is not fine-grained application authorization. Any reachable service still needs its own access policy.

ORIGINAL QUESTION PRESERVED

Source guide p. 34

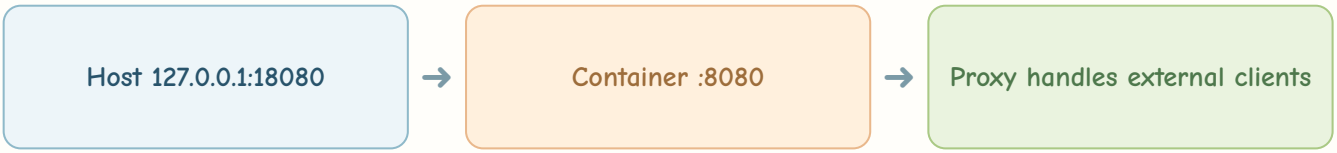
BUILD / RUNTIME

Q048 How do you interpret -p 127.0.0.1:18080:8080?

THE 20-SECOND ANSWER

It maps TCP port 18080 on the engine host's loopback address to port 8080 in the container.

↳ Draw this in the interview



WHY / HOW IT WORKS

The application listens inside on 8080, typically on 0.0.0.0. Omitting the host IP commonly publishes on all host interfaces. For a remote engine, the mapping belongs to the remote host, not the local terminal.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run -d --name dk-q-api \
  -p 127.0.0.1:18080:8080 IMAGE
docker port dk-q-api
# Host client: http://127.0.0.1:18080
```

THE INTERVIEWER MAY ASK…

How would you expose the service through a reverse proxy while restricting direct access?

A. Bind the backend only on a private/loopback interface and publish the reverse proxy's intended listener. Apply firewall, routing, and TLS policies at the proxy.

⚠ WATCH OUT — Loopback publication had an L2 reachability caveat in Engine releases older than 28.0.0. Use a maintained engine and verify the actual network boundary.

ORIGINAL QUESTION PRESERVED

Source guide p. 34

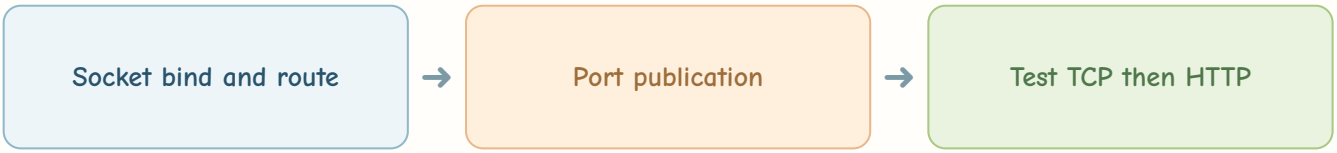
BUILD / RUNTIME

Q049 Why can a published port still return connection refused?

THE 20-SECOND ANSWER

Publication cannot make an absent, incorrectly bound or incorrectly addressed application listener work.

↳ Draw this in the interview



WHY / HOW IT WORKS

Follow the path: host mapping, container target port, listener address, process readiness. If the process is not ready, a correct mapping can still fail temporarily. Authentication or HTTP errors mean a different layer was reached.

COMMAND OR MINI EXAMPLE — from original guide

```
docker logs APP_CONTAINER
docker inspect APP_CONTAINER --format '{{json .NetworkSettings.Ports}}'
# Check actual app port and whether it binds to 0.0.0.0.
# A listener on container 127.0.0.1 may reject mapped traffic.
```

THE INTERVIEWER MAY ASK…

What is the difference between timeout, refused connection and HTTP 503?

A. Refused means a reachable endpoint actively has no listener; timeout suggests a dropped/unreachable path; HTTP 503 means an HTTP server replied but cannot serve.

⚠ WATCH OUT — Do not treat a successful ping as proof of a working HTTP or database connection. Test the required protocol.

ORIGINAL QUESTION PRESERVED

Source guide p. 35

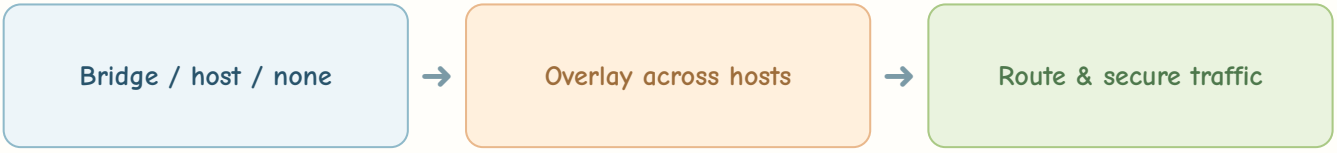
BUILD / RUNTIME

Q050 When would you choose bridge, host, none or overlay networking?

THE 20-SECOND ANSWER

Choose a driver for the needed isolation and reach: bridge for local groups, host for intentional host-stack sharing, none for no normal network, overlay for supported multi-host networking.

↳ Draw this in the interview



WHY / HOW IT WORKS

An offline utility can use none; ordinary local service stacks fit a bridge. Host networking changes port conflict and isolation behavior. Swarm overlay networking connects services across participating engines.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --rm --network none alpine:3.22 \
  sh -c 'cat /proc/net/route'
# Host networking requires a supported platform/configuration.
# A plain bridge does not become a multi-host fabric.
```

THE INTERVIEWER MAY ASK…

What additional routing and security decisions arise across multiple hosts?

A. Across hosts you need overlay/routed connectivity, service discovery, ingress, firewall rules, encrypted traffic, and policies controlling service-to-service access.

⚠ WATCH OUT — Desktop host-network support and operating-system behavior require version-specific checking. Driver choice alone does not provide encryption or a complete network policy.

ORIGINAL QUESTION PRESERVED

Source guide p. 35

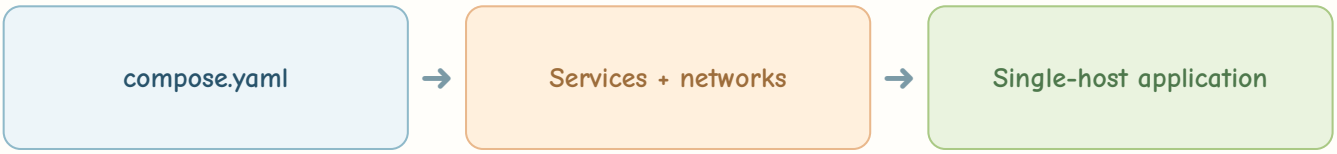
OPERATIONS

Q051 What does Docker Compose do?

THE 20-SECOND ANSWER

Compose describes and manages a related set of services, networks and volumes using a declarative application configuration.

↳ Draw this in the interview



WHY / HOW IT WORKS

Compose creates the required objects and starts the service. A service definition can also declare a build, environment and mounts. Current examples use the Compose plugin command with a space.

COMMAND OR MINI EXAMPLE — from original guide

```
# compose.yaml fragment
services:
  web:
    image: nginx:stable
    ports:
      - "127.0.0.1:18080:80"
# Start with: docker compose up -d
```

THE INTERVIEWER MAY ASK...

What operational responsibilities remain on a single-host Compose deployment?

A. You still manage host availability, backups, upgrades, security patches, monitoring, secrets, capacity, and disaster recovery. Compose is not a full cluster orchestrator.

⚠ WATCH OUT — A Compose service definition is not an automatic multi-host scheduler or high-availability plan. An obsolete top-level version key is not needed in this guide's files.

ORIGINAL QUESTION PRESERVED

Source guide p. 36

OPERATIONS

Q052 Does depends_on guarantee a database is ready?

THE 20-SECOND ANSWER

The basic form orders startup; a health-conditioned dependency can wait for the configured health check to pass.

↳ Draw this in the interview



WHY / HOW IT WORKS

Waiting for PostgreSQL to accept connections reduces an initial race. It does not prove migrations finished, credentials work for the app, or the database will stay available. A completed migration job can use a different dependency condition.

COMMAND OR MINI EXAMPLE — from original guide

```
depends_on:
  db:
    condition: service_healthy
# db must define a meaningful healthcheck.
# API connection retry still handles later database outages.
```

THE INTERVIEWER MAY ASK...

How would you distinguish database readiness from application schema readiness?

A. Database readiness proves the server accepts suitable connections; schema readiness needs a migration/version check verifying the tables and columns the app requires.

⚠ WATCH OUT — Dependency-level restart behavior in Compose is distinct from the container's restart policy. Check what operation triggers each.

ORIGINAL QUESTION PRESERVED

Source guide p. 37

OPERATIONS

Q053 What is the difference between up, start, restart and down in Compose?

THE 20-SECOND ANSWER

up reconciles the application and can recreate services; start starts existing containers; restart restarts them; down removes the project's runtime objects.

↳ Draw this in the interview



WHY / HOW IT WORKS

Editing an environment value in YAML and running restart does not recreate the container with the new configuration. up evaluates the desired definition; --build asks it to build eligible service images.

COMMAND OR MINI EXAMPLE — from original guide

```
docker compose up -d --build
docker compose ps
docker compose logs --tail=100 web
# After a configuration change, normally use up again.
```

THE INTERVIEWER MAY ASK...

What command would you use after changing a port mapping?

A. Run `docker compose up -d` so Compose recreates services when configuration changes. `restart` does not generally apply new port mappings.

⚠ WATCH OUT — down normally preserves named volumes; down -v requests volume removal. A restart does not pull or rebuild a new image by itself.

ORIGINAL QUESTION PRESERVED

Source guide p. 37

OPERATIONS

Q054 How does Compose configuration interpolation differ from container environment?

THE 20-SECOND ANSWER

Compose substitutes variables while reading YAML; service environment and env_file determine variables passed into the container.

↳ Draw this in the interview



WHY / HOW IT WORKS

A project .env file commonly supplies interpolation values, but it does not automatically inject every value into every container. $$ can defer a dollar sign to a later container shell where appropriate.

COMMAND OR MINI EXAMPLE — from original guide

```
services:
  web:
    image: nginx:stable
    environment:
      APP_LABEL: "${APP_LABEL:-local}"
# Inspect the resolved model: docker compose config
```

THE INTERVIEWER MAY ASK...

Why can a variable be visible to Compose but absent from the application?

A. Compose interpolation reads shell/.env values while rendering configuration. A variable reaches the app only if the service's environment/secrets configuration passes it.

⚠ WATCH OUT — config can print resolved sensitive values. Different sources have defined precedence; inspect the final model in a safe environment.

ORIGINAL QUESTION PRESERVED

Source guide p. 38

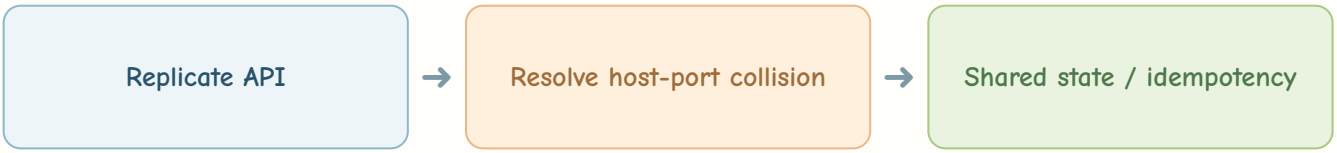
OPERATIONS

Q055 Can you scale a Compose service with fixed names and published ports?

THE 20-SECOND ANSWER

Scaling works only when the configuration avoids collisions such as a fixed container_name or one identical host port for every replica.

↳ Draw this in the interview



WHY / HOW IT WORKS

Replica containers can listen on the same internal port because they have separate network stacks. The host cannot bind one address/port pair three times. A proxy provides a stable client endpoint and chooses a replica.

COMMAND OR MINI EXAMPLE — from original guide

```
docker compose up -d --scale api=3
# api must allow replicas: no fixed container_name.
# Put replicas behind a proxy instead of sharing one fixed host bind.
```

THE INTERVIEWER MAY ASK...

How would you handle HTTP sessions and scheduled jobs across three API replicas?

A. Use shared session storage or stateless tokens, and protect scheduled jobs with leader election, partitioned ownership, or idempotent processing.

⚠ WATCH OUT — Scaling a database service does not configure database replication. Horizontal capacity also depends on shared state and downstream limits.

ORIGINAL QUESTION PRESERVED

Source guide p. 38

OPERATIONS

Q056 Why should one image serve multiple environments?

THE 20-SECOND ANSWER

Promoting the same tested artifact separates code identity from environment-specific configuration and reduces rebuild drift.

↳ Draw this in the interview



WHY / HOW IT WORKS

The executable content stays fixed while configuration changes. Environment-specific values can include database endpoints and feature policy. Runtime configuration must still be validated before the service accepts traffic.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run -e SPRING_PROFILES_ACTIVE=test APP_IMAGE
docker run -e SPRING_PROFILES_ACTIVE=prod APP_IMAGE
# These are separate deployment alternatives, not one stack.
```

THE INTERVIEWER MAY ASK…

Which configuration values should be recorded in release evidence?

A. Record the image digest, env/config version, secret references, resource limits, networks, mounts, health policy, and migration version for every release.

⚠ WATCH OUT — A prod profile name is not an access-control mechanism. Startup overrides can change behavior dramatically even with an identical digest.

ORIGINAL QUESTION PRESERVED

Source guide p. 39

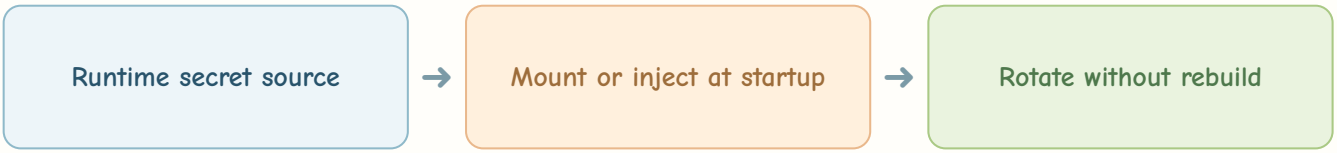
OPERATIONS

Q057 Are environment variables a secure secret store?

THE 20-SECOND ANSWER

They are a convenient configuration channel, but authorized inspection, process tooling and accidental logging can expose their values.

↳ Draw this in the interview



WHY / HOW IT WORKS

Using a file avoids typing every value in the command, but the resulting variables remain part of container configuration. Threat modeling must include who can inspect the daemon and the process.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --env-file local-settings.env IMAGE
# Appropriate for disposable local values with restricted file access.
# Prefer a platform secret facility for real deployment credentials.
```

THE INTERVIEWER MAY ASK...

How would you rotate a secret without publishing a new application image?

A. Place the new secret in a runtime secret provider or mounted file, then restart/refresh affected services safely. The application image need not change.

⚠ WATCH OUT — Do not paste docker inspect output containing secrets into support tickets. An env file still needs access control and must stay out of source and build context.

ORIGINAL QUESTION PRESERVED

Source guide p. 39

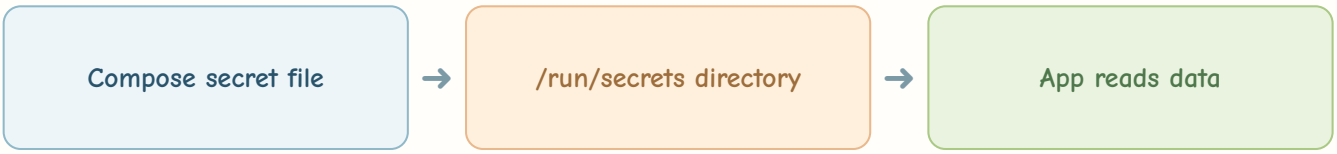
OPERATIONS

Q058 How do Compose secrets reach a service?

THE 20-SECOND ANSWER

A granted secret is typically made available as a mounted file; the application must explicitly read that file or support a corresponding file-based setting.

↳ Draw this in the interview



WHY / HOW IT WORKS

The PostgreSQL image understands its _FILE option and reads the mounted content. That is an image-specific contract. Compose does not turn every arbitrary secret name into an environment variable automatically.

COMMAND OR MINI EXAMPLE — from original guide

```
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
secrets:
  db_password:
    file: ./secrets/db_password.txt
```

THE INTERVIEWER MAY ASK...

How can a Spring application consume a mounted secret directory?

A. Mount the secret directory and configure Spring Boot to read config-tree properties, for example `spring.config.import=optional:configtree:/run/secrets/`.

⚠ WATCH OUT — Local Compose file-backed secrets are not the same as Swarm's managed encrypted secret distribution. Protect the source file and the engine host.

ORIGINAL QUESTION PRESERVED

Source guide p. 40

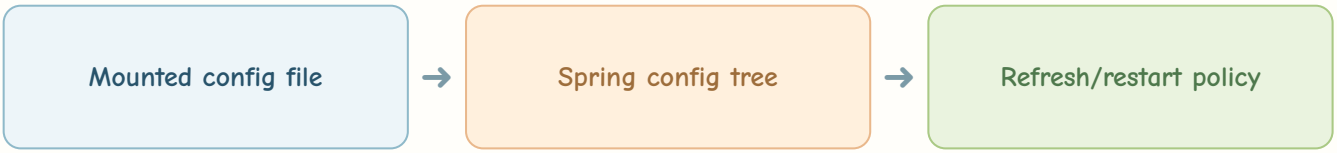
OPERATIONS

Q059 How can Spring Boot read file-mounted configuration secrets?

THE 20-SECOND ANSWER

A supported Spring Boot configuration tree import maps files in a directory into configuration properties.

↳ Draw this in the interview



WHY / HOW IT WORKS

The trailing slash marks a directory import. The optional prefix permits a missing directory, which can be useful locally but may be wrong for mandatory production credentials. Choose file names that map to the intended properties.

COMMAND OR MINI EXAMPLE — from original guide

```
# Runtime environment value:
SPRING_CONFIG_IMPORT=optional:configtree:/run/secrets/
# Example mounted file:
# /run/secrets/spring.datasource.password
# File contents become that property's value.
```

THE INTERVIEWER MAY ASK...

Would file replacement immediately refresh every Spring configuration property?

A. No. File replacement does not automatically refresh every bound property or live database connection. Use a supported refresh strategy or rolling restart.

⚠ WATCH OUT — A missing secret should fail startup when required. Restrict endpoint exposure and logs so configuration values are not disclosed.

ORIGINAL QUESTION PRESERVED

Source guide p. 40

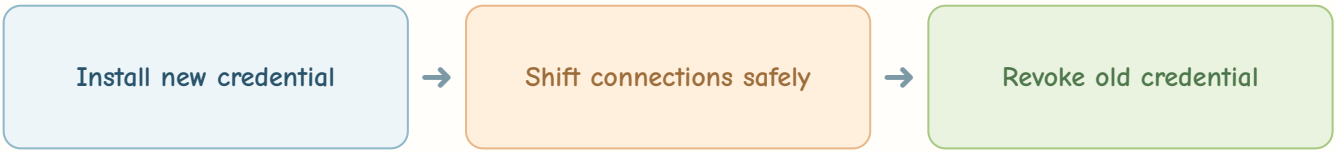
OPERATIONS

Q060 How should secrets be rotated in a containerized service?

THE 20-SECOND ANSWER

Coordinate secret delivery, application reload or recreation, credential overlap and verification; replacing bytes alone is not a universal reload mechanism.

↳ Draw this in the interview



WHY / HOW IT WORKS

A connection pool may retain old authenticated sessions after a file changes. A staged rotation avoids a sudden outage and lets you observe whether new connections use the intended credential. The exact sequence depends on the credential provider.

COMMAND OR MINI EXAMPLE — from original guide

```
1. Create a new credential with a controlled overlap window.
2. Deliver it through the runtime secret mechanism.
3. Reload or recreate consumers according to their contract.
4. Verify new connections, then revoke the old credential.
```

THE INTERVIEWER MAY ASK...

How would you test rotation while the database has active traffic?

A. Introduce new credentials with an overlap window, rotate one replica at a time, test reads/writes and connection recovery, then revoke the old secret.

⚠ WATCH OUT — Avoid assuming every bind-mounted file update or Compose secret change is hot-reloaded. Do not leave both credentials valid indefinitely.

ORIGINAL QUESTION PRESERVED

Source guide p. 41

OPERATIONS

Q061 What does a Docker HEALTHCHECK do?

THE 20-SECOND ANSWER

It executes a probe and records a health status independently of whether the container's main process is running.

↳ Draw this in the interview



WHY / HOW IT WORKS

Consecutive probe failures can mark the container unhealthy while its process keeps running. Inspect probe output to distinguish a failing service from a missing curl executable or incorrect endpoint.

COMMAND OR MINI EXAMPLE — from original guide

```
# Only if curl exists in this image:
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -fsS http://127.0.0.1:8080/health || exit 1
docker inspect APP_CONTAINER --format '{{json .State.Health}}'
```

THE INTERVIEWER MAY ASK...

Who routes traffic away from an unhealthy instance in your deployment?

A. Docker records health status but does not automatically remove the instance from an external load balancer. Compose/orchestrator/proxy must consume readiness or health appropriately.

⚠ WATCH OUT — A Docker health status by itself does not automatically restart an ordinary standalone container. A controller must act if that behavior is required.

ORIGINAL QUESTION PRESERVED

Source guide p. 42

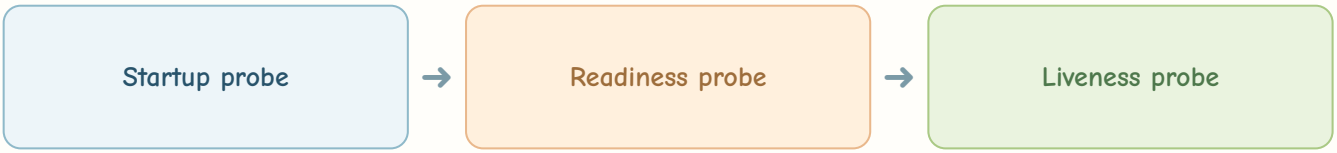
OPERATIONS

Q062 How are liveness, readiness and startup checks different?

THE 20-SECOND ANSWER

Liveness asks whether restart can help; readiness asks whether traffic should be admitted; startup allows initialization before normal failure decisions.

↳ Draw this in the interview



WHY / HOW IT WORKS

An unavailable shared database may make an API unready without making every JVM dead. Restarting all instances during a dependency outage can worsen recovery. A slow first boot needs a realistic initialization budget.

COMMAND OR MINI EXAMPLE — from original guide

```
Startup:   has initial boot completed?
Liveness:  is the process making progress?
Readiness: can this instance serve the required traffic?
# These are application/platform concepts, not three Dockerfile probes.
```

THE INTERVIEWER MAY ASK...

What should readiness return during graceful shutdown?

A. Readiness should become false before accepting new work is stopped, so the router drains traffic while current requests can finish.

⚠ WATCH OUT — One Docker HEALTHCHECK cannot express every orchestration policy. Actuator endpoint availability and access rules must be configured and verified.

ORIGINAL QUESTION PRESERVED

Source guide p. 43

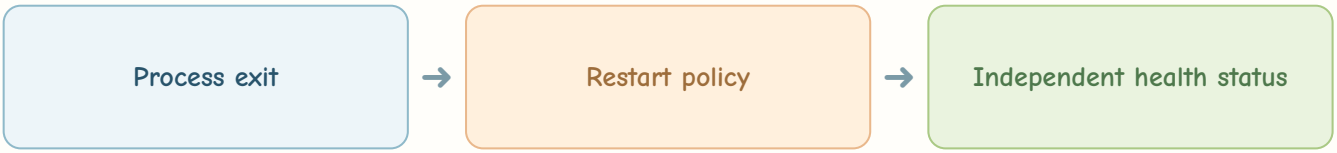
OPERATIONS

Q063 How do no, on-failure, always and unless-stopped restart policies differ?

THE 20-SECOND ANSWER

They define restart behavior for exits and daemon lifecycle events, with important handling for intentional stops.

↳ Draw this in the interview



WHY / HOW IT WORKS

on-failure targets nonzero exits; always and unless-stopped provide broader restart behavior. Their treatment of a deliberately stopped container across daemon restarts differs. They operate on exit state rather than HTTP readiness.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run -d --restart=unless-stopped \
  --name dk-q-managed IMAGE
docker inspect dk-q-managed \
  --format '{{json .HostConfig.RestartPolicy}}'
```

THE INTERVIEWER MAY ASK...

Why could a container be unhealthy with a restart count of zero?

A. A failed HEALTHCHECK changes health status, not necessarily process exit. Docker restart policies react to container stopping, not health status alone.

⚠ WATCH OUT — A restart policy is not multi-host failover and is unsuitable for a batch job that must not repeat side effects. Apply backoff and diagnosis at the right layer.

ORIGINAL QUESTION PRESERVED

Source guide p. 43

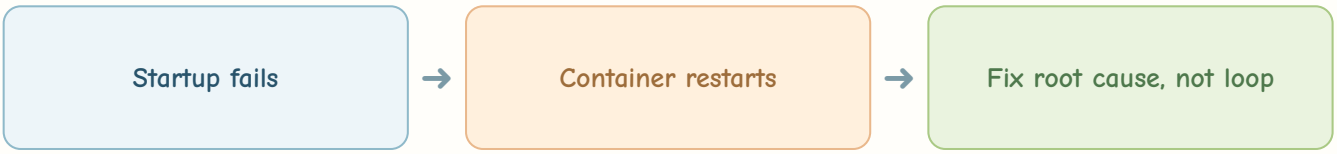
OPERATIONS

Q064 Why do services get stuck in a restart loop?

THE 20-SECOND ANSWER

A deterministic startup failure repeats because restart policy changes process lifecycle, not its invalid inputs or broken dependencies.

↳ Draw this in the interview



WHY / HOW IT WORKS

A missing JAR, invalid JVM flag or required secret can make each attempt fail identically. Preserve evidence, repair the configuration or artifact, and recreate with the intended inputs.

COMMAND OR MINI EXAMPLE — from original guide

```
docker logs --tail=100 APP_CONTAINER
docker inspect APP_CONTAINER \
  --format 'exit={{.State.ExitCode}} restarts={{.RestartCount}}'
# Inspect mounts, command, credentials and required dependencies.
```

THE INTERVIEWER MAY ASK...

What should a migration job do if the schema is already updated?

A. A migration should be idempotent: check its applied version, verify state, and exit successfully if the required schema change already exists.

⚠ WATCH OUT — Repeated restarts can overwrite logs, consume CPU and repeat non-idempotent startup work. Do not raise retry counts as the only repair.

ORIGINAL QUESTION PRESERVED

Source guide p. 44

OPERATIONS

Q065 What makes graceful deployment work end to end?

THE 20-SECOND ANSWER

Withdraw new traffic, allow in-flight work to finish within a budget, handle worker shutdown, then terminate and verify the replacement.

↳ Draw this in the interview



WHY / HOW IT WORKS

Correct signal forwarding is necessary but insufficient. Proxy connections, request durations, Kafka acknowledgments and transaction boundaries all influence the outcome. Deployment verification must cover user-visible errors and unfinished work.

COMMAND OR MINI EXAMPLE — from original guide

```
Traffic controller: stop admitting new requests
Application: drain accepted work and close resources
Runtime: deliver signal; enforce final timeout
Deployment: confirm healthy replacement and observed behavior
```

THE INTERVIEWER MAY ASK...

How would you test shutdown during an order request that publishes an event?

A. Start an order request, trigger shutdown, and verify the accepted request completes or is safely retried; inspect outbox records to prevent a lost event.

⚠ WATCH OUT — A zero-downtime HTTP rollout cannot promise zero message redelivery or exactly-once external effects. Design idempotency and recovery explicitly.

ORIGINAL QUESTION PRESERVED

Source guide p. 44

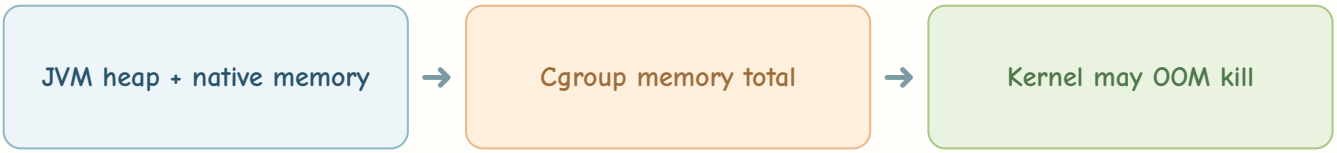
OPERATIONS

Q066 Why can a container be OOM-killed even when Java has free heap?

THE 20-SECOND ANSWER

The memory budget includes heap plus native allocations, thread stacks, metaspace, direct buffers and other charged memory, not just live Java objects.

↳ Draw this in the interview



WHY / HOW IT WORKS

The remaining space must cover the actual non-heap workload and any other processes. A process can be killed by the kernel before Java can produce a heap dump or OutOfMemoryError. Measure peak memory under representative load.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --memory=512m \
  -e JAVA_TOOL_OPTIONS='-Xms128m -Xmx320m' IMAGE
# Illustrative starting point: leaves 192 MiB outside max heap.
```

THE INTERVIEWER MAY ASK...

Which evidence separates a Java heap failure from a cgroup memory kill?

A. Compare JVM OOM exceptions/heap dumps with `docker inspect` OOMKilled, cgroup memory.events, RSS/native memory and host/kernel logs.

⚠ WATCH OUT — The example is a budget sketch, not a safe universal ratio. Page cache and runtime accounting also affect observed numbers.

ORIGINAL QUESTION PRESERVED

Source guide p. 45

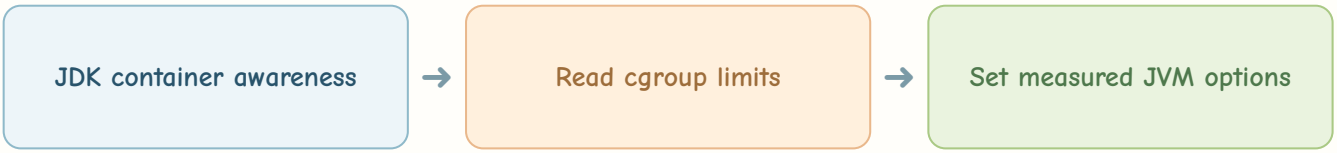
OPERATIONS

Q067 Do modern JVMs understand container resource limits?

THE 20-SECOND ANSWER

Supported modern JDKs on Linux detect relevant container CPU and memory constraints, but exact behavior depends on JDK update, cgroup setup and flags.

↳ Draw this in the interview



WHY / HOW IT WORKS

Inspect the JVM's reported limits and heap settings inside the constrained container. This is more reliable than assuming an old Java 8 build behaves like a current Java 21 runtime.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --rm --memory=512m --cpus=1 \
  eclipse-temurin:21-jre \
  java -XshowSettings:system -XshowSettings:vm -version
```

THE INTERVIEWER MAY ASK...

How would you compare the exact JDK updates used in development and production?

A. Log `java -version` and inspect exact image digests in both places; compare `UseContainerSupport`, heap defaults, CPU count and patch-level fixes.

⚠ WATCH OUT — Host resource detection and container detection can differ across versions. Do not disable container support without understanding the resulting sizing.

ORIGINAL QUESTION PRESERVED

Source guide p. 46

OPERATIONS

Q068 How do -Xmx and MaxRAMPercentage differ?

THE 20-SECOND ANSWER

Xmx explicitly caps heap size; MaxRAMPercentage participates in ergonomic heap sizing from the JVM's detected available memory when an explicit heap maximum is not supplied.

↳ Draw this in the interview



WHY / HOW IT WORKS

A percentage can adapt one image to several memory limits; an explicit cap gives a direct budget. Small-memory heuristics and other flags can affect the ergonomic result, so verify actual sizing rather than multiplying blindly.

COMMAND OR MINI EXAMPLE — from original guide

```
JAVA_TOOL_OPTIONS='-XX:MaxRAMPercentage=60'
# Alternative explicit policy:
JAVA_TOOL_OPTIONS='-Xmx320m'
# Inspect effective MaxHeapSize on the selected JDK.
```

THE INTERVIEWER MAY ASK...

What changes if a health probe starts a second JVM in the same container?

A. Two JVMs share one container budget. A health JVM adds heap and native memory, so size the main JVM to leave room for probes and other processes.

⚠ WATCH OUT — Neither option caps all process memory. Setting heap near the container limit can leave too little room for native use.

ORIGINAL QUESTION PRESERVED

Source guide p. 46

OPERATIONS

Q069 What is the difference between --cpus, --cpu-shares and --cpuset-cpus?

THE 20-SECOND ANSWER

They represent CPU bandwidth limiting, relative scheduling weight and allowed CPU placement respectively, subject to platform and cgroup support.

↳ Draw this in the interview



WHY / HOW IT WORKS

A 1.5-CPU quota permits a limited amount of scheduled CPU time rather than guaranteeing two dedicated cores. Placement chooses eligible cores; weight influences competition. Inspect throttling when latency grows at high load.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --cpus=1.5 IMAGE
# Separate alternative: --cpuset-cpus=0,1
# Relative weight example: --cpu-shares=512
```

THE INTERVIEWER MAY ASK...

Why might adding more application threads worsen latency under a small quota?

A. Under CPU quota, too many runnable threads add context switching and throttling; queues and p99 latency can grow despite more threads.

⚠ WATCH OUT — CPU shares are not a hard ceiling. Quotas can affect tail latency and GC even when host-wide utilization looks modest.

ORIGINAL QUESTION PRESERVED

Source guide p. 47

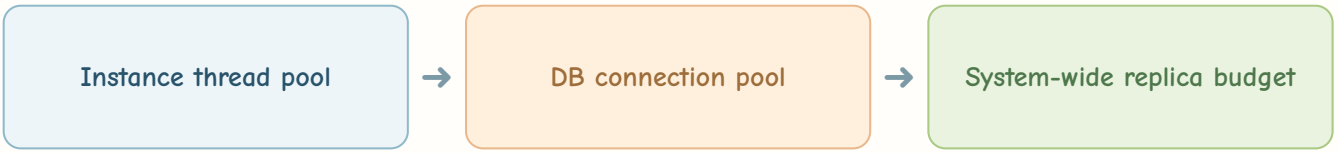
OPERATIONS

Q070 How should thread pools and connection pools be sized in a container?

THE 20-SECOND ANSWER

Use measured workload, CPU budget, blocking behavior and downstream capacity; do not size every pool from unconstrained host hardware.

↳ Draw this in the interview



WHY / HOW IT WORKS

A modest per-instance pool can collectively overload a database after scaling. Virtual threads improve waiting-task economics but do not create extra CPU or unlimited database connections. Bound expensive concurrency separately.

COMMAND OR MINI EXAMPLE — from original guide

```
API replicas: 4
DB pool per replica: 25
Potential app connections: 4 x 25 = 100
# Add migration jobs, administration and failover headroom.
```

THE INTERVIEWER MAY ASK...

How would you choose a safe maximum replica count from a database connection limit?

A. Reserve database capacity for admin work and other consumers; divide the safe connection budget by max pool size per replica, then include headroom.

⚠ WATCH OUT — A single benchmark number cannot establish production capacity. Measure throughput, tail latency, queueing, throttling and dependency saturation.

ORIGINAL QUESTION PRESERVED

Source guide p. 47

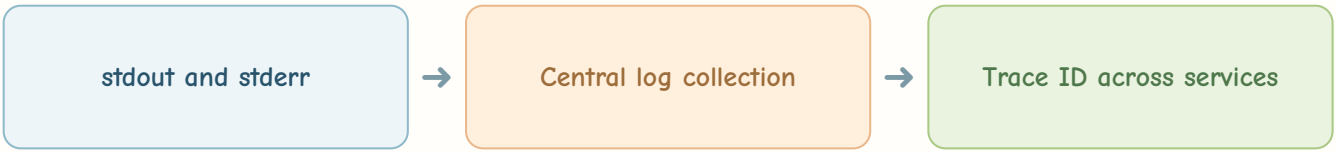
OPERATIONS

Q071 Where should a containerized application write logs?

THE 20-SECOND ANSWER

Usually to stdout and stderr, with runtime log collection, retention and external aggregation configured separately.

↳ Draw this in the interview



WHY / HOW IT WORKS

The logging driver captures supported container output. Central aggregation enables searching across recreated replicas. The local driver provides rotation behavior; json-file deployments should explicitly manage rotation and disk use.

COMMAND OR MINI EXAMPLE — from original guide

```
docker logs --since=10m --tail=200 APP_CONTAINER
docker run --log-driver=local IMAGE
# Java application logging: console output with request context.
```

THE INTERVIEWER MAY ASK…

How would you correlate one failed request across several services?

A. Propagate a correlation/trace ID across services, record it in structured logs and tracing spans, and search all systems using that same identifier.

⚠ WATCH OUT — docker logs availability depends on the driver and configuration. File-only application logging is not automatically collected from an arbitrary image path.

ORIGINAL QUESTION PRESERVED

Source guide p. 48

OPERATIONS

Q072 What are inspect, stats, top and events useful for?

THE 20-SECOND ANSWER

They expose configuration and state, live resource use, process information and lifecycle events respectively.

↳ Draw this in the interview



WHY / HOW IT WORKS

Use inspect for actual limits and mounts, stats for a resource snapshot, top for process relationships, and events for transitions. Combine these with application metrics and timestamps rather than interpreting one number in isolation.

COMMAND OR MINI EXAMPLE — from original guide

```
docker inspect APP_CONTAINER
docker stats --no-stream APP_CONTAINER
docker top APP_CONTAINER
docker events --since 10m --filter container=APP_CONTAINER
```

THE INTERVIEWER MAY ASK...

Which additional time-series data would support a CPU-throttling diagnosis?

A. Collect cgroup throttled-period counts/time, container CPU usage, host CPU saturation, request latency, runnable-thread counts and queue depth over time.

⚠ WATCH OUT — On Linux, CLI memory presentation may subtract cache differently from API data. A snapshot can miss a short spike.

ORIGINAL QUESTION PRESERVED

Source guide p. 48

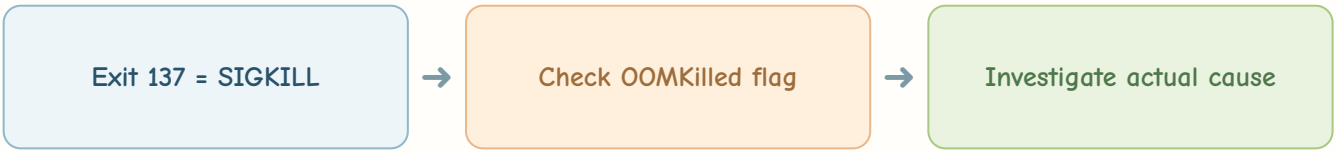
OPERATIONS

Q073 Does exit code 137 always mean an OOM kill?

THE 20-SECOND ANSWER

No. 137 commonly corresponds to SIGKILL termination, which can result from memory enforcement, a forced stop or another kill source.

↳ Draw this in the interview



WHY / HOW IT WORKS

OOMKilled is a useful clue, but diagnosis needs the relevant time window and memory evidence. A stop timeout ending in SIGKILL can also produce 137. Java can independently fail with a managed OutOfMemoryError.

COMMAND OR MINI EXAMPLE — from original guide

```
docker inspect APP_CONTAINER \
  --format 'exit={{.State.ExitCode}} oom={{.State.OOMKilled}}'
docker logs --tail=100 APP_CONTAINER
# Correlate with runtime events and host/cgroup evidence.
```

THE INTERVIEWER MAY ASK...

Why might an OOM kill produce no Java heap dump?

A. The kernel may kill the JVM abruptly without letting it handle a Java `OutOfMemoryError` or write a heap dump.

⚠ WATCH OUT — A process can deliberately exit with a numeric code, so status alone is not a proof of cause. Preserve logs and termination context.

ORIGINAL QUESTION PRESERVED

Source guide p. 49

OPERATIONS

Q074 How do you debug an image that has no shell or a container that has exited?

THE 20-SECOND ANSWER

Use metadata, logs and copied artifacts; use an approved diagnostic image or platform tooling when live process inspection is required.

↳ Draw this in the interview



WHY / HOW IT WORKS

docker cp can retrieve files from a stopped container, unlike exec. For a shell-free running image, diagnostics must match access, PID namespace and tool compatibility constraints. A dedicated debug variant can be useful in test environments.

COMMAND OR MINI EXAMPLE — from original guide

```
docker inspect FAILED_CONTAINER
docker logs FAILED_CONTAINER
docker cp FAILED_CONTAINER:/app/diagnostic.txt ./diagnostic.txt
# Only if the file exists and is appropriate to export.
```

THE INTERVIEWER MAY ASK...

How would you collect a thread dump without permanently expanding the runtime image?

A. Use a debug-capable sidecar/ephemeral container where supported, JFR endpoints, or a controlled JVM diagnostic signal/attach method; keep the production image minimal.

⚠ WATCH OUT — Installing tools into the live production container creates drift. Copied files may contain personal data or credentials.

ORIGINAL QUESTION PRESERVED

Source guide p. 49

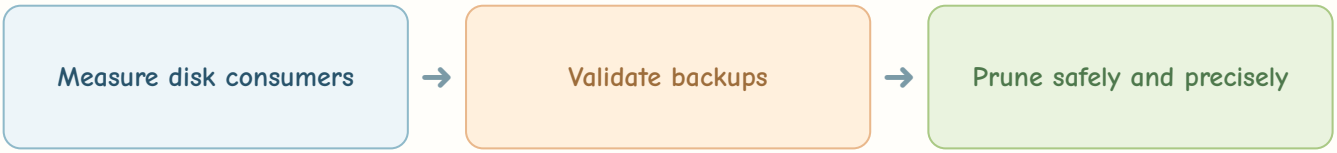
OPERATIONS

Q075 How do you diagnose Docker disk usage before cleanup?

THE 20-SECOND ANSWER

Inspect image, writable-layer, cache, log and volume usage, then remove only identified disposable objects.

↳ Draw this in the interview



WHY / HOW IT WORKS

A large database volume needs a different response from stale build cache. Reclaimable image sizes include sharing relationships, so sum carefully. Retention policies prevent recurring exhaustion better than emergency broad deletion.

COMMAND OR MINI EXAMPLE — from original guide

```
docker system df -v
docker ps -a --size
docker volume ls
docker image ls
# Select explicit obsolete objects after checking ownership and data.
```

THE INTERVIEWER MAY ASK...

Which backup and restore evidence is needed before retiring a database volume?

A. Before deletion, confirm a recent tested backup, verified restore on a separate host, recovery point, credentials, schema compatibility and owner approval.

⚠ WATCH OUT — system prune is destructive, has category-specific rules, and is not a database cleanup strategy. Never assume all unused volumes contain worthless data.

ORIGINAL QUESTION PRESERVED

Source guide p. 50

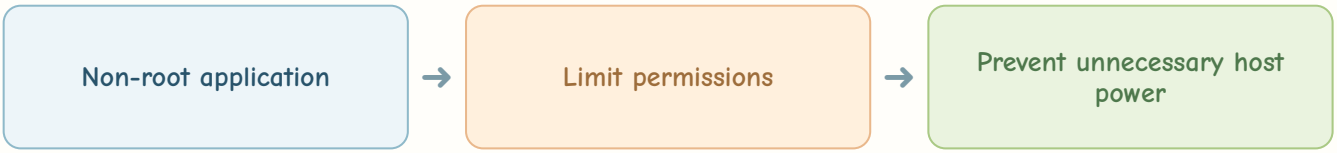
PRODUCTION / SENIOR

Q076 Why should application containers normally avoid root?

THE 20-SECOND ANSWER

A non-root process reduces available authority inside the container and limits some consequences of an application compromise.

↳ Draw this in the interview



WHY / HOW IT WORKS

The application must work with that UID and any required writable directories. Dropping capabilities and restricting privilege acquisition add separate controls. Test the actual workload under the intended policy.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --user 10001:10001 \
  --cap-drop=ALL --security-opt=no-new-privileges \
  IMAGE
```

THE INTERVIEWER MAY ASK...

What explicit privilege would your API need to listen on port 8080?

A. None: port 8080 is unprivileged on normal Linux defaults. Non-root can listen there without adding privileged-port capabilities.

⚠ WATCH OUT — Non-root is not immunity to kernel vulnerabilities, vulnerable application code or overly permissive mounted files. Capabilities and security profiles are distinct.

ORIGINAL QUESTION PRESERVED

Source guide p. 51

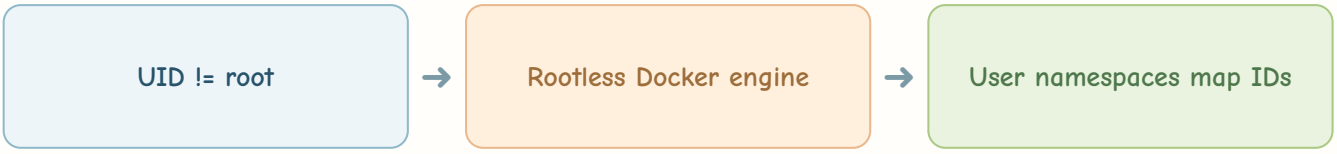
PRODUCTION / SENIOR

Q077 What is the difference between non-root, rootless Docker and user namespace remapping?

THE 20-SECOND ANSWER

They change application identity, daemon authority and host-to-container UID mapping at different layers.

↳ Draw this in the interview



WHY / HOW IT WORKS

These controls can overlap but are not interchangeable. Rootless mode changes daemon-level privileges and can affect supported resource or networking features. Remapping affects how mounted ownership is interpreted on the host.

COMMAND OR MINI EXAMPLE — from original guide

```
Non-root USER:    application UID is not zero
Rootless engine:  daemon and containers run without host root
User remapping:   container IDs map to different host IDs
```

THE INTERVIEWER MAY ASK...

How would rootless operation affect your bind mount and CPU-limit setup?

A. Rootless changes UID mappings, host path access, network behavior, and availability of resource controls. Test bind mounts and cgroups on the actual host.

⚠ WATCH OUT — Setting USER in a Dockerfile does not make a rootful engine rootless. Inspect runtime capabilities on the actual supported platform.

ORIGINAL QUESTION PRESERVED

Source guide p. 52

PRODUCTION / SENIOR

Q078 Why is mounting the Docker socket into a container dangerous?

THE 20-SECOND ANSWER

Engine API access can grant broad control over containers and host resources within that engine's authority.

↳ Draw this in the interview



WHY / HOW IT WORKS

A job with a rootful engine socket can often create privileged workloads or mount sensitive host paths. Making the socket bind read-only does not turn the API into a read-only API. Rootless engines still expose the owning user's authority.

COMMAND OR MINI EXAMPLE — from original guide

```
# Review this sensitive mount in CI definitions:
# /var/run/docker.sock:/var/run/docker.sock
# Prefer isolated builders and carefully bounded credentials.
```

THE INTERVIEWER MAY ASK...

What separate trust boundary would you choose for pull-request image builds?

A. Run untrusted pull-request builds on isolated workers without production registry credentials or privileged daemon access. Use short-lived scoped tokens only for trusted jobs.

⚠ WATCH OUT — A convenient build agent must not receive the production engine socket by default. Isolation between trusted and untrusted jobs matters.

ORIGINAL QUESTION PRESERVED

Source guide p. 52

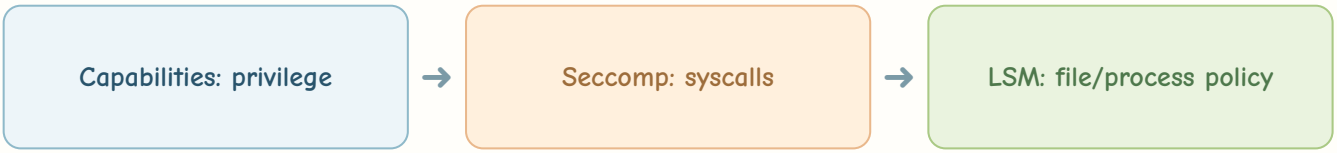
PRODUCTION / SENIOR

Q079 What do capabilities, seccomp and AppArmor/SELinux control?

THE 20-SECOND ANSWER

Capabilities divide privileged operations, seccomp filters system calls, and mandatory access controls constrain operations through host policy.

↳ Draw this in the interview



WHY / HOW IT WORKS

These mechanisms address different paths to abuse. If the application fails, identify the required operation before adding a narrowly justified capability or policy change. Keep host kernel and runtime patches current.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --cap-drop=ALL \
  --security-opt=no-new-privileges \
  IMAGE
# Retain the platform's supported default seccomp policy.
```

THE INTERVIEWER MAY ASK...

How would you investigate a denied syscall without disabling all confinement?

A. Check audit/denial logs, seccomp profiles and effective capabilities. Temporarily reproduce with a narrowly adjusted policy, then permit only the needed operation.

⚠ WATCH OUT — --privileged or an unconfined policy removes important protections and is not a general fix for permission errors. Host policy can differ across environments.

ORIGINAL QUESTION PRESERVED

Source guide p. 53

PRODUCTION / SENIOR

Q080 Can a container run with a read-only filesystem?

THE 20-SECOND ANSWER

Yes, when the application has explicit writable mounts for its real requirements and can avoid writing elsewhere.

↳ Draw this in the interview



WHY / HOW IT WORKS

The image filesystem cannot be changed through ordinary writes, while /tmp supplies bounded scratch space. Logs can go to stdout. Persistent uploads or caches require explicitly designed storage rather than arbitrary writes under /app.

COMMAND OR MINI EXAMPLE — from original guide

```
docker run --read-only \
  --tmpfs /tmp:rw,size=64m \
  --user 10001:10001 --cap-drop=ALL IMAGE
```

THE INTERVIEWER MAY ASK...

Which startup behavior would you test before enforcing this policy on Spring Boot?

A. Test temp-file creation, Java extracted libraries, logs, cache writes and startup scripts. Provide specific writable tmpfs/volume paths such as `/tmp` if required.

⚠ WATCH OUT — A read-only root filesystem does not make every mounted volume read-only. Audit actual write paths, native-library extraction and JVM temporary files.

ORIGINAL QUESTION PRESERVED

Source guide p. 53

PRODUCTION / SENIOR

Q081 Why should base images be maintained and periodically rebuilt?

THE 20-SECOND ANSWER

Application security depends on runtime and OS components too; an unchanged application can still require an updated base and retesting.

↳ Draw this in the interview



WHY / HOW IT WORKS

Pinning provides a stable input; an update process prevents that stable input from becoming indefinitely stale. Rebuild, scan and test the new artifact, then record the new release digest.

COMMAND OR MINI EXAMPLE — from original guide

```
FROM eclipse-temurin:21-jre
# Production policy can pin a reviewed digest.
# An update job proposes the new digest and runs verification.
```

THE INTERVIEWER MAY ASK...

How would you roll out a base-only patch across many Java services?

A. Update the approved base digest, rebuild all dependent services in CI, scan/test images, and roll out gradually with rollback digests recorded.

⚠ WATCH OUT — A smaller image is not automatically supported or vulnerability-free. Deleting package managers does not patch vulnerable runtime libraries.

ORIGINAL QUESTION PRESERVED

Source guide p. 54

PRODUCTION / SENIOR

Q082 What is an SBOM, and what does it tell you?

THE 20-SECOND ANSWER

A software bill of materials inventories detected components and versions; it supports assessment but is not proof that an image is secure.

↳ Draw this in the interview



WHY / HOW IT WORKS

An inventory helps determine whether an affected package may be present. Detection coverage differs for OS packages, shaded Java dependencies and custom binaries. Pair the inventory with vulnerability information and actual usage analysis.

COMMAND OR MINI EXAMPLE — from original guide

```
docker buildx build --sbom=true \
  --tag REGISTRY/APP:BUILD_ID --push .
# Consume the attached inventory with your assessment tools.
```

THE INTERVIEWER MAY ASK…

How would you assess a CVE affecting a transitive Java dependency?

A. Check the vulnerable package and exact version in the SBOM, actual runtime reachability, exploit conditions, fixed version, and transitive upgrade path.

⚠ WATCH OUT — An absent finding can reflect scanner limitations. An SBOM does not establish a publisher's identity or that the application passed tests.

ORIGINAL QUESTION PRESERVED

Source guide p. 54

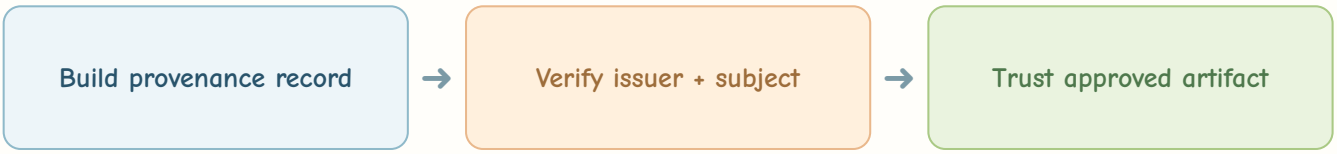
PRODUCTION / SENIOR

Q083 What is build provenance?

THE 20-SECOND ANSWER

It records information about how an artifact was produced, such as builder, inputs and build parameters, according to the chosen format and mode.

↳ Draw this in the interview



WHY / HOW IT WORKS

Provenance supports tracing a release back to source and build inputs. Its usefulness depends on a trusted builder and verifiable binding to the image digest. Richer records can also disclose more build details.

COMMAND OR MINI EXAMPLE — from original guide

```
docker buildx build --provenance=mode=max \
  --tag REGISTRY/APP:BUILD_ID --push .
# Review the evidence attached to the published artifact.
```

THE INTERVIEWER MAY ASK...

What makes an attestation trustworthy rather than merely present?

A. Validate a trusted signature and attestation issuer, authorized build identity, subject digest, source revision and policy. An arbitrary attached JSON file is not enough.

⚠ WATCH OUT — Build arguments can appear in provenance. Credentials belong in secret mechanisms, not ARG. Attestation support also depends on export and storage configuration.

ORIGINAL QUESTION PRESERVED

Source guide p. 55

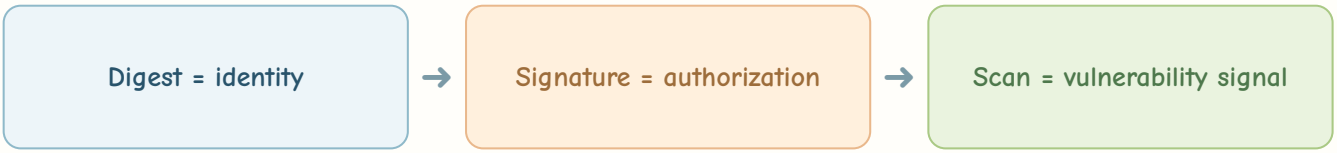
PRODUCTION / SENIOR

Q084 How are image signatures different from digests and vulnerability scans?

THE 20-SECOND ANSWER

A digest identifies content, a signature can authenticate an assertion about that content, and a scan assesses known issues under a specific policy and database.

↳ Draw this in the interview



WHY / HOW IT WORKS

Each answers a different question. Signature verification must check the expected identity and trust policy, not merely find a signature. A legitimate publisher can still publish vulnerable or broken software.

COMMAND OR MINI EXAMPLE — from original guide

```
Content check:   compare approved image digest
Publisher check: verify signature identity and policy
Risk check:      inspect findings and allowed exceptions
Runtime check:   test application behavior and access policy
```

THE INTERVIEWER MAY ASK...

Which evidence would you retain when approving a temporary vulnerability exception?

A. Store CVE details, exploitability analysis, mitigation, business risk owner, exception expiry, evidence links and a planned update date.

⚠ WATCH OUT — Signing technology and registry integration vary. A deployment policy must enforce verification rather than rely on a developer remembering a manual step.

ORIGINAL QUESTION PRESERVED

Source guide p. 55

PRODUCTION / SENIOR

Q085 Why can deleting a secret in a later Dockerfile step still leak it?

THE 20-SECOND ANSWER

Earlier image layers or build output may retain the original bytes even when the final filesystem view hides the file.

↳ Draw this in the interview



WHY / HOW IT WORKS

Layered storage means a later deletion does not rewrite all prior content. The response to a real exposure includes revoking the secret and correcting the build pipeline, not just removing the final visible file.

COMMAND OR MINI EXAMPLE — from original guide

```
# Unsafe pattern:
# COPY credentials.txt /tmp/credentials.txt
# RUN use-credentials && rm /tmp/credentials.txt
# Use a build secret mount and prevent output leakage instead.
```

THE INTERVIEWER MAY ASK...

How would you investigate where the exposed image and credential were distributed?

A. List affected image digests, registries, tags, build logs and deployed clusters; revoke the leaked credential, rotate it, and search for unauthorized access.

⚠ WATCH OUT — Private registries do not make embedded long-lived credentials an acceptable design. Intermediate caches and logs also need review.

ORIGINAL QUESTION PRESERVED

Source guide p. 56

PRODUCTION / SENIOR

Q086 What should a container CI pipeline verify?

THE 20-SECOND ANSWER

Source tests, build inputs, the produced image's startup and behavior, security policy and release identity all need appropriate checks.

↳ Draw this in the interview



WHY / HOW IT WORKS

A JAR passing unit tests can still fail because the runtime lacks a library, uses the wrong UID or receives the wrong arguments. Image tests should exercise the packaging and runtime contract, not only repeat compilation.

COMMAND OR MINI EXAMPLE — from original guide

```
Source commit -> unit/integration tests -> image build
Image candidate -> runtime smoke test -> assessment
Published digest -> deployment -> observed verification
```

THE INTERVIEWER MAY ASK...

What smoke test catches a wrong container port or missing certificate bundle?

A. Boot the exact built image under test settings, call the expected HTTP health endpoint, and confirm SSL/TLS connections or CA trust to a test dependency.

⚠ WATCH OUT — A pipeline step named test proves nothing without the assertion and failure policy. Verify exit codes and readiness deadlines.

ORIGINAL QUESTION PRESERVED

Source guide p. 57

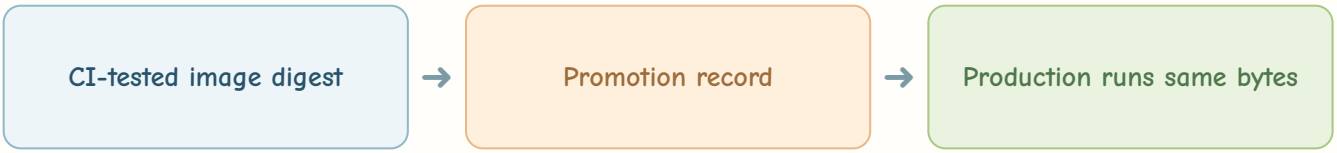
PRODUCTION / SENIOR

Q087 Why deploy the same digest that passed image testing?

THE 20-SECOND ANSWER

It ties production deployment to the verified artifact and avoids rebuild differences between stages.

↳ Draw this in the interview



WHY / HOW IT WORKS

Source, dependencies, timestamped output or a base tag can change when rebuilding. Digest promotion removes that specific drift while preserving environment-specific values. Record which platform image or index was approved.

COMMAND OR MINI EXAMPLE — from original guide

```
Test:   registry.example.com/team/app@sha256:APP_DIGEST
Stage:  same digest + stage configuration
Prod:   same digest + production configuration
```

THE INTERVIEWER MAY ASK…

How would you prove which digest an incident-affected replica actually ran?

A. Inspect orchestrator-reported image IDs/digests and `docker inspect` for each replica, then compare them with the signed release manifest.

⚠ WATCH OUT — Production behavior can still differ because configuration, architecture, data and dependencies differ. Test the relevant variant and environment contract.

ORIGINAL QUESTION PRESERVED

Source guide p. 58

PRODUCTION / SENIOR

Q088 How should image-build credentials be handled in CI?

THE 20-SECOND ANSWER

Provide short-lived or narrowly scoped credentials through protected CI channels, with separate permissions for reading, building and release publication.

↳ Draw this in the interview



WHY / HOW IT WORKS

The source of a build can execute code during dependency resolution and compilation. An external pull request must not inherit a trusted release job's secrets or engine access. Limit credentials by destination and purpose.

COMMAND OR MINI EXAMPLE — from original guide

```
Untrusted PR: read-only dependencies; no production secrets
Trusted release: authorized publish identity
Deployment: permission to deploy an approved digest
```

THE INTERVIEWER MAY ASK...

How would you test contributions from forks while protecting a private registry?

A. Build fork contributions without secrets or write permissions. Run trusted publishing only after review on protected branches/workflows.

⚠ WATCH OUT — Hiding a secret in logs does not stop malicious code from transmitting it elsewhere. Trust separation must occur before executing the job.

ORIGINAL QUESTION PRESERVED

Source guide p. 58

PRODUCTION / SENIOR

Q089 What is the difference between rolling, blue-green and canary deployment?

THE 20-SECOND ANSWER

Rolling replaces instances incrementally; blue-green switches between complete environments; canary admits limited traffic to a candidate before expansion.

↳ Draw this in the interview



WHY / HOW IT WORKS

All need readiness, capacity, version compatibility and a rollback decision. Blue-green may require more temporary resources. Canary decisions need meaningful traffic and user-impact signals, not only an alive process.

COMMAND OR MINI EXAMPLE — from original guide

```
Rolling:    old and new replicas overlap
Blue-green: two deployment groups; controlled traffic switch
Canary:     measured small candidate share, then promotion
```

THE INTERVIEWER MAY ASK...

What metrics would stop a canary before most users are affected?

A. Stop rollout if error rates, p95/p99 latency, saturation, business failures or health checks breach preset thresholds relative to baseline.

⚠ WATCH OUT — Container tooling alone does not provide every traffic strategy. Shared database changes can prevent a simple application rollback.

ORIGINAL QUESTION PRESERVED

Source guide p. 59

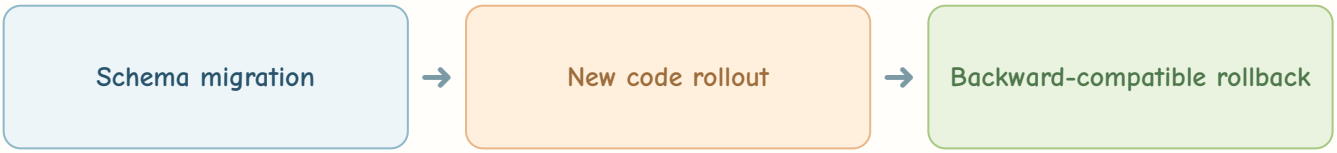
PRODUCTION / SENIOR

Q090 Why do database migrations complicate container rollback?

THE 20-SECOND ANSWER

Rolling back an image does not automatically restore the schema, data or external effects changed by the newer release.

↳ Draw this in the interview



WHY / HOW IT WORKS

An expand-contract strategy gives overlapping application versions a compatible period. A migration job needs ownership, retries and a clear completion result. Destructive transformations require a separate recovery plan.

COMMAND OR MINI EXAMPLE — from original guide

```
Expand: add compatible schema without removing old fields
Deploy: allow old and new code to coexist safely
Migrate: backfill and verify
Contract: remove old paths after the rollback window closes
```

THE INTERVIEWER MAY ASK...

How would you roll back code after a non-reversible data transformation?

A. Roll code forward with a compatibility adapter or restore transformed data from a validated backup. A simple old-image rollback cannot reverse changed data.

⚠ WATCH OUT — A container restart is not a migration lock. Running every replica's startup migration concurrently can create contention or failure.

ORIGINAL QUESTION PRESERVED

Source guide p. 59

PRODUCTION / SENIOR

Q091 What is a multi-platform image?

THE 20-SECOND ANSWER

It uses an image index or manifest list to reference distinct image variants for supported operating-system and CPU combinations.

↳ Draw this in the interview



WHY / HOW IT WORKS

The same readable tag can lead an AMD64 host and an ARM64 host to different manifests. The variants should provide equivalent application behavior but their binary layers need not be identical.

COMMAND OR MINI EXAMPLE — from original guide

```
docker buildx imagetools inspect REGISTRY/APP:TAG
# Example variants: linux/amd64 and linux/arm64
# A pull normally selects a compatible variant for the target.
```

THE INTERVIEWER MAY ASK…

What exactly does pinning the index digest fix?

A. Pinning the index digest locks the manifest list and which platform variants it references, but different hosts still select their respective platform-specific manifests.

⚠ WATCH OUT — Attestation descriptors can also appear in an index. Do not mistake every descriptor for a runnable platform variant.

ORIGINAL QUESTION PRESERVED

Source guide p. 60

PRODUCTION / SENIOR

Q092 How do you build and publish AMD64 and ARM64 variants?

THE 20-SECOND ANSWER

Use a builder configured for the platforms through native workers, supported emulation or a deliberate cross-compilation approach.

↳ Draw this in the interview



WHY / HOW IT WORKS

The resulting registry reference can contain both variants. The Java runtime base must support the platforms, and native dependencies must be compatible. Exporting directly to a registry avoids assuming the local image store supports every multi-platform output.

COMMAND OR MINI EXAMPLE — from original guide

```
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --tag REGISTRY/APP:BUILD_ID --push .
```

THE INTERVIEWER MAY ASK...

When is a native ARM worker preferable to emulation?

A. Native ARM workers avoid emulation overhead and can test native libraries on real ARM behavior, especially for expensive builds and platform-specific tests.

⚠ WATCH OUT — The command does not provision missing builders or emulators. Test each target variant; a successful build is not a compatibility test.

ORIGINAL QUESTION PRESERVED

Source guide p. 60

PRODUCTION / SENIOR

Q093 Why does exec format error happen?

THE 20-SECOND ANSWER

An executable can have an incompatible architecture or format, or a script can lack a valid interpreter contract.

↳ Draw this in the interview



WHY / HOW IT WORKS

Compare the image's declared platform with the host and the file being launched. A manually copied AMD64 binary can still be wrong inside an ARM64 image. Script failures can also stem from CRLF or a missing interpreter.

COMMAND OR MINI EXAMPLE — from original guide

```
docker image inspect IMAGE \
  --format '{{.Os}}/{{.Architecture}}'
# Check copied native binaries, script shebang and line endings.
# Also confirm the interpreter actually exists in the image.
```

THE INTERVIEWER MAY ASK...

How would you prevent a developer's native build output entering a cross-platform image?

A. Build dependencies inside the selected target architecture or cross-compile with verified target flags. Never blindly COPY a developer's host-built native executable.

⚠ WATCH OUT — A platform label does not validate every copied file. permission denied is a related but different failure that needs permission and mount checks.

ORIGINAL QUESTION PRESERVED

Source guide p. 61

PRODUCTION / SENIOR

Q094 Can Java bytecode portability hide OS-library incompatibility?

THE 20-SECOND ANSWER

Yes. JNI, native agents, TLS providers and other libraries can depend on architecture, libc and available system components.

↳ Draw this in the interview



WHY / HOW IT WORKS

A smaller Alpine-based image uses a different libc family from many conventional Linux bases. Pure Java code can run while a native transport or agent fails at load time. Exercise the actual features under the target image.

COMMAND OR MINI EXAMPLE — from original guide

```
Java class: portable within its supported Java runtime contract
Native .so: requires matching architecture and ABI
Base choice: verify glibc/musl and required library support
```

THE INTERVIEWER MAY ASK...

Which integration test would detect a failing native compression library?

A. Run a container integration test on each target architecture that loads the compression library and performs actual compress/decompress with checked output.

⚠ WATCH OUT — Do not claim Alpine is always wrong or always better. Choose from supported dependencies and operational needs, then measure size and performance.

ORIGINAL QUESTION PRESERVED

Source guide p. 61

PRODUCTION / SENIOR

Q095 How do BUILDPLATFORM and TARGETPLATFORM help cross-platform builds?

THE 20-SECOND ANSWER

They identify the builder's execution platform and requested output platform, enabling explicit control of stages and compiler targets.

↳ Draw this in the interview



WHY / HOW IT WORKS

A native build stage can run on the builder while producing an artifact for the target, if the toolchain actually supports that cross-compilation. Automatic platform arguments must be declared where their values are used in stage instructions.

COMMAND OR MINI EXAMPLE — from original guide

```
# Dockerfile sketch, not a complete cross compiler:
FROM --platform=$BUILDPLATFORM TOOLCHAIN_IMAGE AS build
ARG TARGETPLATFORM
# Build the target artifact using a toolchain that supports it.
FROM RUNTIME_IMAGE
```

THE INTERVIEWER MAY ASK...

Why can a Dockerfile run quickly but produce the wrong native artifact?

A. A fast build might accidentally compile for BUILDPLATFORM and copy that output into a TARGETPLATFORM image. Confirm target architecture and run it in a target test.

⚠ WATCH OUT — Merely printing TARGETPLATFORM does not cross-compile a binary. Java-native packaging and generated libraries still need a target-aware workflow.

ORIGINAL QUESTION PRESERVED

Source guide p. 62

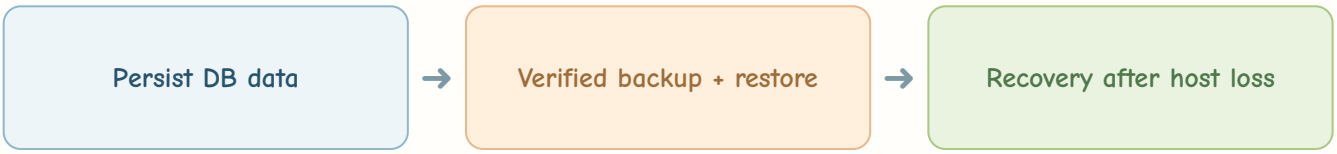
PRODUCTION / SENIOR

Q096 Can you run a database in Docker safely?

THE 20-SECOND ANSWER

Yes, with a supported image, correctly mounted durable storage, resource planning, backup/restore verification and an operational recovery policy.

↳ Draw this in the interview



WHY / HOW IT WORKS

Separating data from the container lets you recreate the process without discarding the database. Database upgrades still require compatible data handling. Single-host storage remains dependent on that host's reliability.

COMMAND OR MINI EXAMPLE — from original guide

```
# PostgreSQL 17 image-specific data path:
--mount type=volume,src=orders-db,dst=/var/lib/postgresql/data
# Do not publish its port unless a host client needs access.
# Review data path and upgrade guidance for the chosen major version.
```

THE INTERVIEWER MAY ASK...

How would you recover after losing both the container and its host disk?

A. Restore a tested off-host backup and transaction logs onto replacement storage, recreate the database container, verify integrity, then bring clients back carefully.

⚠ WATCH OUT — A volume is not a replica or an off-host backup. PostgreSQL 18+ image layout differs, so do not blindly reuse a 17 mount recipe.

ORIGINAL QUESTION PRESERVED

Source guide p. 63

PRODUCTION / SENIOR

Q097 Why are raw volume copies risky for a running database?

THE 20-SECOND ANSWER

A filesystem copy can capture an inconsistent point unless the database and storage snapshot procedure provide the required consistency.

↳ Draw this in the interview



WHY / HOW IT WORKS

A logical backup tool understands database consistency better than copying arbitrary live files. The example covers one database; roles, global objects and point-in-time recovery require additional planning. Avoid a TTY for streamed backup bytes.

COMMAND OR MINI EXAMPLE — from original guide

```
docker exec DB_CONTAINER \
  pg_dump -U app -d orders > orders.sql
# Example logical backup; authenticate through an approved mechanism.
# Validate by restoring into a separate disposable database.
```

THE INTERVIEWER MAY ASK...

When would physical backup plus transaction-log archival be preferable?

A. Use physical backups plus WAL/archive logs for large databases, low RPO and point-in-time recovery, when the database's tooling supports it.

⚠ WATCH OUT — A successful dump command is not a tested restore. Preserve encryption, retention, integrity and recovery-time requirements.

ORIGINAL QUESTION PRESERVED

Source guide p. 63

PRODUCTION / SENIOR

Q098 Why do database initialization environment variables seem to stop working?

THE 20-SECOND ANSWER

Many official database images use initialization settings only when creating an empty data directory, not to alter an existing database on every start.

↳ Draw this in the interview



WHY / HOW IT WORKS

Retained volume data changes the startup path. Recreating the container with a new password value does not imply the existing PostgreSQL role password has changed. Inspect the image contract and the actual database state.

COMMAND OR MINI EXAMPLE — from original guide

```
# First initialization: POSTGRES_PASSWORD_FILE is consumed.
# Later start with the same volume: existing database remains.
# Change real roles/credentials using the database's own procedure.
```

THE INTERVIEWER MAY ASK...

How would you diagnose a password mismatch without deleting any data?

A. Check whether a data directory was initialized earlier, inspect effective env/secret inputs and logs, then use supported SQL/password rotation without deleting the volume.

⚠ WATCH OUT — Deleting the volume to make settings take effect is destructive. Treat credential rotation and database migrations as explicit operations.

ORIGINAL QUESTION PRESERVED

Source guide p. 64

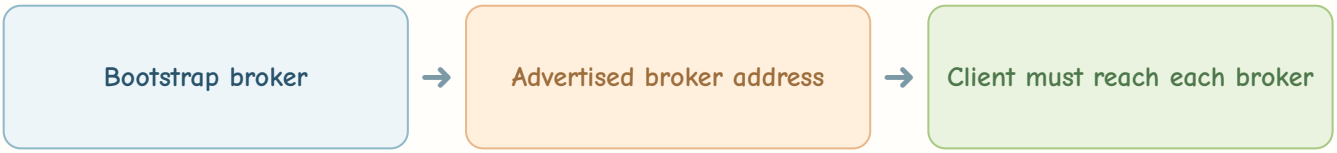
PRODUCTION / SENIOR

Q099 Why does Kafka connect initially and then fail from a container?

THE 20-SECOND ANSWER

The bootstrap endpoint may be reachable while broker metadata advertises addresses that the client cannot resolve or reach.

↳ Draw this in the interview



WHY / HOW IT WORKS

Kafka clients discover broker endpoints after bootstrapping. Each client network needs usable advertised addresses. A successful TCP connection to the initial bootstrap address therefore proves only the first step.

COMMAND OR MINI EXAMPLE — from original guide

```
Bootstrap from API container: kafka:9092
Advertised internal listener: kafka:9092
Host client listener: a separately reachable host endpoint
# Image-specific environment names must follow that image's contract.
```

THE INTERVIEWER MAY ASK...

How would you support both host tools and same-network Spring clients?

A. Configure Kafka advertised listeners separately for host clients and Docker-network clients; each client must receive broker endpoints reachable from its own network.

⚠ WATCH OUT — listener configuration, TLS identity and advertised endpoint design must agree. Docker port mapping does not rewrite arbitrary broker metadata for every client network.

ORIGINAL QUESTION PRESERVED

Source guide p. 64

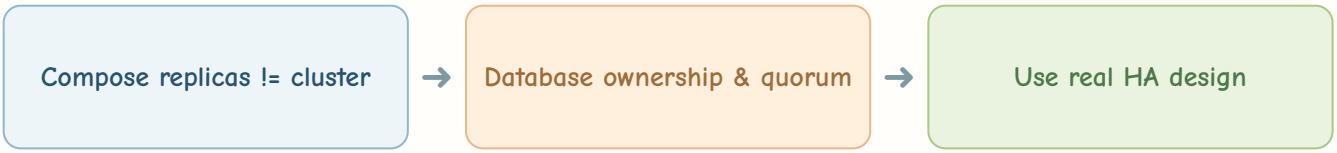
PRODUCTION / SENIOR

Q100 Can docker compose scale turn one database into a database cluster?

THE 20-SECOND ANSWER

No. Starting replicas of a container does not configure application replication, membership, leader election or data ownership.

↳ Draw this in the interview



WHY / HOW IT WORKS

Each database process must have the correct identity and storage. Pointing unrelated replicas at the same data directory can corrupt data or violate the database's supported usage. Managed stateful services may simplify the responsibility.

COMMAND OR MINI EXAMPLE — from original guide

```
docker compose up -d --scale api=3
# Appropriate only if the API's state and jobs permit it.
# Database replication requires its own explicit cluster design.
```

THE INTERVIEWER MAY ASK...

Which workload would you keep outside a first single-host Compose deployment?

A. Keep a critical production database, distributed Kafka cluster, or other stateful service needing multi-host failover outside an initial single-host setup.

⚠ WATCH OUT — Likewise, several Kafka or Redis containers do not automatically form a correctly configured fault-tolerant cluster. Test failure and recovery semantics.

ORIGINAL QUESTION PRESERVED

Source guide p. 65

PRODUCTION / SENIOR

Q101 How do Engine, Compose, Swarm and Kubernetes differ?

THE 20-SECOND ANSWER

Engine runs container objects; Compose manages an application definition; Swarm and Kubernetes add cluster-level service or workload orchestration.

↳ Draw this in the interview



WHY / HOW IT WORKS

A cluster controller can replace failed workloads on eligible nodes, subject to capacity and storage constraints. Compose can be useful for development and suitable single-host services without becoming a cluster scheduler.

COMMAND OR MINI EXAMPLE — from original guide

```
Single engine: container lifecycle on its selected host
Compose:       related services, config, networks and volumes
Cluster:       scheduling, desired replicas and platform recovery
```

THE INTERVIEWER MAY ASK…

What requirement would make you move beyond a single host?

A. Multi-node recovery, rolling rescheduling after host failure, stronger deployment controls, or independent autoscaling can justify adopting an orchestrator.

⚠ WATCH OUT — Product selection should follow requirements and team operations capacity. A cluster is not automatically simpler or highly available in every component.

ORIGINAL QUESTION PRESERVED

Source guide p. 66

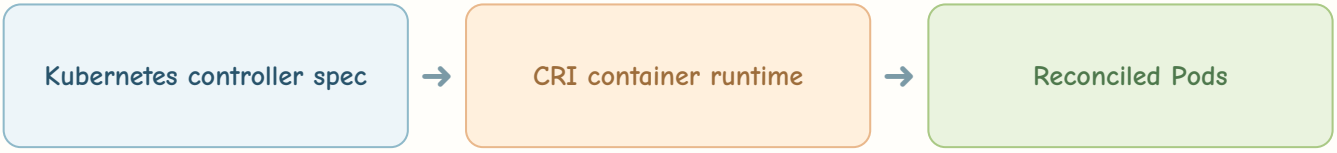
PRODUCTION / SENIOR

Q102 Does Kubernetes require Docker Engine or dockershim?

THE 20-SECOND ANSWER

Current Kubernetes uses CRI-compatible runtimes; Docker Engine needs an adapter if selected as the runtime, while Docker-built compatible images remain usable.

↳ Draw this in the interview



WHY / HOW IT WORKS

Image format and node runtime integration are separate contracts. Removing dockershim did not make Dockerfiles unusable. Docker Engine itself does not implement the Kubernetes CRI contract directly.

COMMAND OR MINI EXAMPLE — from original guide

```
Docker build -> compatible image in registry
Kubernetes -> CRI runtime such as containerd -> workload
# The old in-tree dockershim was removed in Kubernetes 1.24.
```

THE INTERVIEWER MAY ASK...

Why might editing a managed container directly be undone by its controller?

A. A Deployment controller continuously reconciles desired state. A manual change to one Pod/container can disappear when it is replaced from the declared template.

⚠ WATCH OUT — Do not treat Docker CLI commands as the workload-control interface for a Kubernetes-managed service. Use the orchestrator's ownership model.

ORIGINAL QUESTION PRESERVED

Source guide p. 67

PRODUCTION / SENIOR

Q103 Why should a scalable API avoid local durable state?

THE 20-SECOND ANSWER

Replacement or rescheduling can lose container-local data, and independent replicas cannot safely assume the same local files or memory.

↳ Draw this in the interview



WHY / HOW IT WORKS

Keeping durable state behind explicit services allows replicas to come and go. Local caches may still be useful when invalidation and loss are acceptable. Background jobs need coordination and idempotency rather than one in-memory singleton per replica.

COMMAND OR MINI EXAMPLE — from original guide

```
Durable orders -> database
Shared uploads -> appropriate object/file storage
Session policy -> external state or suitable signed tokens
Local /tmp -> disposable scratch data
```

THE INTERVIEWER MAY ASK...

How would you prevent three replicas from charging the same order twice?

A. Use a shared durable idempotency store with a unique order/payment key and an atomic business transaction; each replica follows the same contract.

⚠ WATCH OUT — Stateless HTTP handlers can still create stateful external effects. The architecture must define retry and concurrency behavior.

ORIGINAL QUESTION PRESERVED

Source guide p. 67

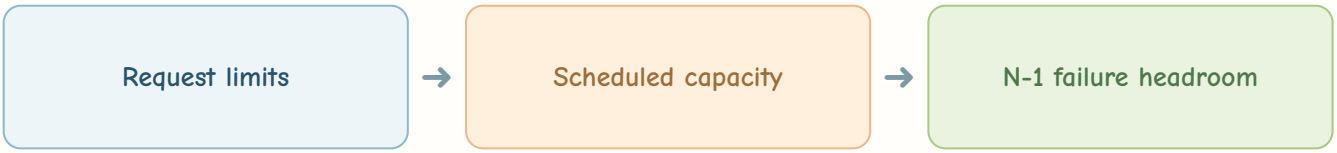
PRODUCTION / SENIOR

Q104 Do resource limits guarantee a workload gets scheduled or has reserved capacity?

THE 20-SECOND ANSWER

No. Runtime ceilings, scheduling requests or reservations, available capacity and failure headroom solve different resource questions.

↳ Draw this in the interview



WHY / HOW IT WORKS

A memory limit can protect against growth while insufficient cluster capacity prevents replacement elsewhere. An orchestrator's requests or reservations guide placement according to its own policy. Load testing determines whether the admitted workload meets latency goals.

COMMAND OR MINI EXAMPLE — from original guide

```
Runtime ceiling: prevent unrestricted per-workload consumption
Placement budget: determine whether the node can accept work
Failure headroom: permit recovery after a node is lost
```

THE INTERVIEWER MAY ASK...

What capacity would remain if your largest node failed?

A. Plan N-1 capacity: after the largest node fails, remaining nodes must still fit critical requests plus headroom or degrade according to an explicit policy.

⚠ WATCH OUT — A nominal replica count does not prove survivability during node loss. Storage location and network dependencies can constrain failover too.

ORIGINAL QUESTION PRESERVED

Source guide p. 68

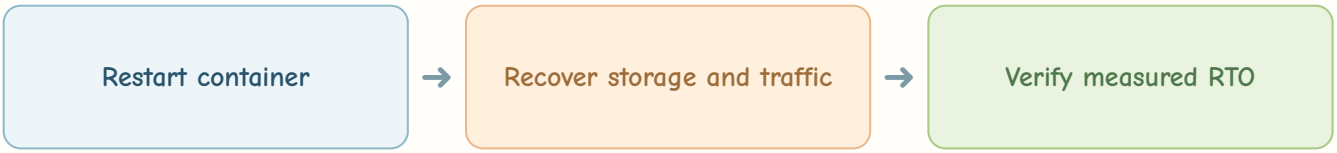
PRODUCTION / SENIOR

Q105 What are the limits of container-level high availability?

THE 20-SECOND ANSWER

A restarted or replaced process restores only part of service availability; traffic, durable data, dependency health and recovery time still matter.

↳ Draw this in the interview



WHY / HOW IT WORKS

Choose acceptable outage and data-loss objectives first, then prove them with failure exercises. A replica may start correctly but be unable to reach data or receive user traffic. Observe both infrastructure and user-visible behavior.

COMMAND OR MINI EXAMPLE — from original guide

```
API replica loss: admit traffic to another ready instance
Host loss:        schedule elsewhere with accessible inputs
Data loss:        restore or fail over under a database policy
Dependency loss:  timeouts, degraded behavior and recovery
```

THE INTERVIEWER MAY ASK…

How would you demonstrate the service's recovery time instead of merely describing it?

A. Perform a timed failure rehearsal: kill a node/container, measure alert-to-recovery and request success, then compare observed RTO/RPO to targets.

⚠ WATCH OUT — Three containers on one host share a host failure domain. Replication without independent failure domains can provide less resilience than expected.

ORIGINAL QUESTION PRESERVED

Source guide p. 68

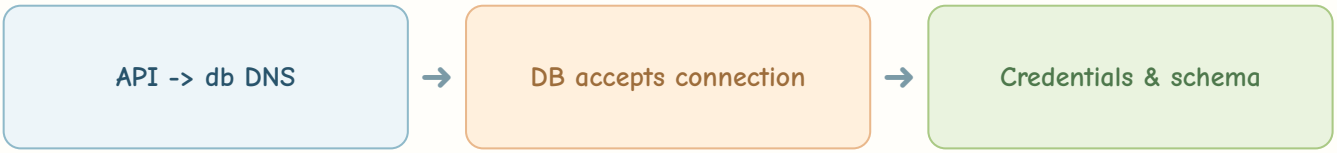
PRODUCTION / SENIOR

Q106 A Spring API cannot reach PostgreSQL in Compose. What do you check first?

THE 20-SECOND ANSWER

Check caller-relative addressing, shared network membership, actual database readiness, credentials and schema compatibility before changing unrelated flags.

↳ Draw this in the interview



WHY / HOW IT WORKS

localhost in the API points to that API, not db. If db resolves but rejects authentication, DNS is already working. If SQL fails because a table is missing, the network may be fine and migration state is the problem.

COMMAND OR MINI EXAMPLE — from original guide

```
Expected app URL: jdbc:postgresql://db:5432/orders
docker compose ps
docker compose logs --tail=100 api db
docker compose config
```

THE INTERVIEWER MAY ASK...

Which error message would move your investigation from network to database state?

A. Errors such as `relation does not exist`, missing table/column or failed migration shift focus from DNS to schema state; authentication failures shift to credentials.

⚠ WATCH OUT — config and logs can include sensitive values. Preserve the current data volume; rebuilding or deleting it is not an initial network diagnosis.

ORIGINAL QUESTION PRESERVED

Source guide p. 69

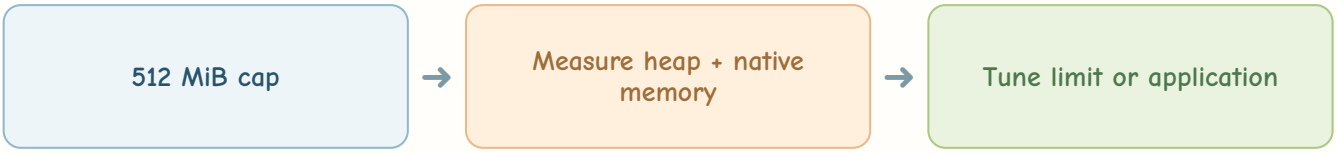
PRODUCTION / SENIOR

Q107 The API works locally but dies under a 512 MiB production limit. What is your approach?

THE 20-SECOND ANSWER

Compare exact image and configuration, verify termination cause, inspect total memory and workload concurrency, then tune from measured evidence.

↳ Draw this in the interview



WHY / HOW IT WORKS

A laptop without limits can hide native or thread-related memory growth. Increasing the limit may be a valid immediate mitigation, but it does not identify the growth source. Separate heap retention from native buffers, thread stacks and cache behavior.

COMMAND OR MINI EXAMPLE — from original guide

```
docker inspect API_CONTAINER \
  --format 'oom={{.State.OOMKilled}} memory={{.HostConfig.Memory}}'
docker stats --no-stream API_CONTAINER
# Compare effective JVM settings and peak-load telemetry.
```

THE INTERVIEWER MAY ASK…

What evidence would justify a larger limit rather than a code or pool-sizing change?

A. Show a stable measured peak including heap, native/thread memory and headroom, with legitimate throughput exceeding the old budget; rule out leaks and oversized pools.

⚠ WATCH OUT — Exit 137 alone is not enough evidence. Do not set Xmx equal to the full container budget as a blanket fix.

ORIGINAL QUESTION PRESERVED

Source guide p. 69

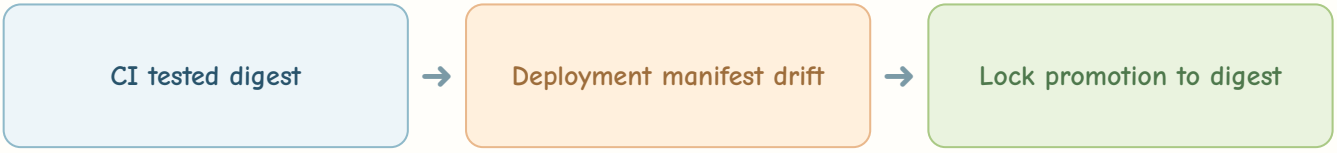
PRODUCTION / SENIOR

Q108 Production runs different code from the image tested in CI. How do you investigate?

THE 20-SECOND ANSWER

Trace source, built artifact, published digest, platform selection and deployed runtime identity; then inspect mounts and configuration that can override packaged behavior.

↳ Draw this in the interview



WHY / HOW IT WORKS

A mutable tag, rebuild or bind mount over /app can change what executes. A stale replica may still use a previous artifact. Resolve each identity step before concluding that the container runtime is inconsistent.

COMMAND OR MINI EXAMPLE — from original guide

```
CI record: source commit + tests + published digest
Runtime:   selected image identity and mounted paths
Registry:  tag history or immutable release record
```

THE INTERVIEWER MAY ASK...

How would you prevent this mismatch in the release process?

A. Allow only tested immutable digests into deployment manifests, promote a signed release record, and reject build/rebuild operations during promotion.

⚠ WATCH OUT — Digest agreement does not prove identical environment configuration or external dependencies. Check both artifact and runtime inputs.

ORIGINAL QUESTION PRESERVED

Source guide p. 70

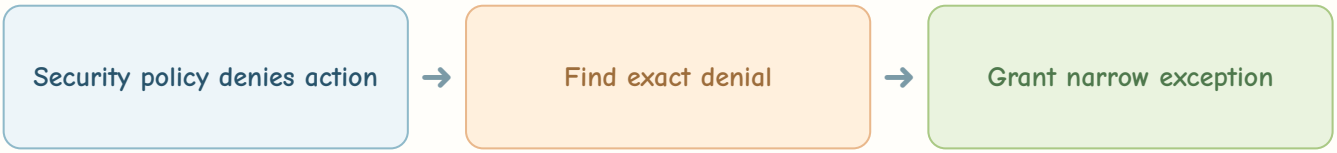
PRODUCTION / SENIOR

Q109 A security policy blocks a previously working Java image. How do you repair it?

THE 20-SECOND ANSWER

Identify the precise denied operation and required path, then adapt the application or grant the narrowest justified allowance.

↳ Draw this in the interview



WHY / HOW IT WORKS

The application may only need a writable scratch directory rather than root privileges. Verify the repaired image with the intended security profile and representative startup and runtime features. Preserve denial evidence for the policy owner.

COMMAND OR MINI EXAMPLE — from original guide

```
Policy: non-root + read-only root + dropped capabilities
Check: temp writes, log paths, native extraction, mount ownership
Repair example: bounded writable /tmp plus stdout logging
```

THE INTERVIEWER MAY ASK…

How would you prove no unexpected persistent write paths remain?

A. Run the app with a read-only root filesystem, trace filesystem write attempts during startup and representative traffic, and whitelist only required writable mounts.

⚠ WATCH OUT — Disabling all controls trades a diagnosis problem for a larger authority boundary. Do not use --privileged as a standard production repair.

ORIGINAL QUESTION PRESERVED

Source guide p. 70

PRODUCTION / SENIOR

Q110 How would you design container delivery for a Spring order service?

THE 20-SECOND ANSWER

Define the artifact, configuration, resource, data, traffic and recovery contracts, then test their interactions rather than stopping at a Dockerfile.

↳ Draw this in the interview



WHY / HOW IT WORKS

Start from traffic and durability requirements. Keep database credentials outside the image, persistent data outside the writable layer, and asynchronous effects retryable and idempotent. Choose single-host or cluster operations from the required failure tolerance.

COMMAND OR MINI EXAMPLE — from original guide

```
Image: tested JAR + maintained Java runtime + non-root UID
Runtime: secrets, bounded memory/CPU, explicit mounts
Service: readiness, graceful drain, metrics and logs
Release: digest promotion + compatible migration + rollback evidence
```

THE INTERVIEWER MAY ASK…

What would you demonstrate in a deployment and failure rehearsal for this design?

A. Deploy a tested immutable image, demonstrate canary metrics and graceful shutdown, kill a replica mid-order, retry the request and verify one order/payment and durable event; restore data from backup.

⚠ WATCH OUT — Docker does not create business correctness or exactly-once external behavior. The service contract must cover duplicates, partial failures and data recovery.

ORIGINAL QUESTION PRESERVED

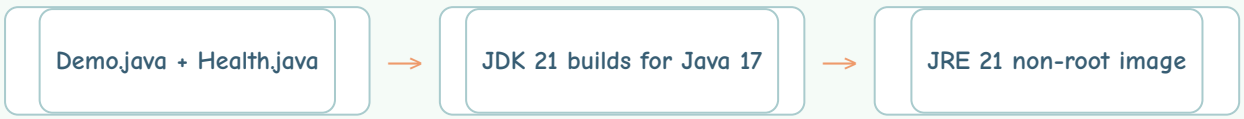
Source guide p. 71

04 / PRACTICAL JAVA MINI PROJECT

One container-ready Java HTTP service

Build a tiny Java HTTP server, give it a Java-based health probe, and package it with a multi-stage Dockerfile. This is a source-derived example; it is not a full Spring Boot application.

Build and runtime contract



Health and shutdown



On SIGTERM, the server's shutdown hook stops accepting new work and releases its listener.

> **Run safely:** Java source compiles with Java 17. The Docker build and run steps require a maintained Docker installation and were not executed in this workspace.

Files in the companion ZIP

Demo.java · Health.java · Dockerfile · .dockerignore · 12 standalone Bash labs · README

JAVA WORKED EXAMPLE / FILE 1 OF 4

Demo.java

HTTP server that returns hello and /health; registers a shutdown hook.

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.concurrent.CountDownLatch;
public class Demo {
    public static void main(String[] args) throws Exception {
        int port = Integer.parseInt(
            System.getenv().getOrDefault("PORT", "8080"));
        HttpServer server = HttpServer.create(
            new InetSocketAddress("0.0.0.0", port), 0);
        server.createContext("/", exchange -> {
            String path = exchange.getRequestURI().getPath();
            String text = path.equals("/health") ? "UP\n" : "hello\n";
            byte[] body = text.getBytes(StandardCharsets.UTF_8);
            exchange.getResponseHeaders().set(
                "Content-Type", "text/plain; charset=utf-8");
            exchange.sendResponseHeaders(200, body.length);
            try (var out = exchange.getResponseBody()) { out.write(body); }
        });
        CountDownLatch stopped = new CountDownLatch(1);
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            System.out.println("draining");
            server.stop(2);
            stopped.countDown();
        }));
        server.start();
        System.out.println("listening on " + port);
        stopped.await();
    }
}
```

Keep the exact files in one directory before running Lab 07. Build inputs, runtime files and health behavior are intentionally separate.

JAVA WORKED EXAMPLE / FILE 2 OF 4

Health.java

A short-timeout Java HTTP probe that exits 0 for healthy and 1 for unhealthy.

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
public class Health {
    public static void main(String[] args) {
        try {
            String port = System.getenv().getOrDefault("PORT", "8080");
            var client = HttpClient.newBuilder()
                .connectTimeout(Duration.ofSeconds(2)).build();
            var request = HttpRequest.newBuilder(
                URI.create("http://127.0.0.1:" + port + "/health"))
                .timeout(Duration.ofSeconds(3)).GET().build();
            var response = client.send(request,
                HttpResponse.BodyHandlers.ofString());
            boolean up = response.statusCode() == 200
                && response.body().equals("UP\n");
            System.out.println(up ? "healthy" : "unhealthy");
            System.exit(up ? 0 : 1);
        } catch (Exception failure) {
            System.out.println("unhealthy");
            System.exit(1);
        }
    }
}
```

> Keep the exact files in one directory before running Lab 07. Build inputs, runtime files and health behavior are intentionally separate.

JAVA WORKED EXAMPLE / FILE 3 OF 4

Dockerfile

Build classes in the JDK stage and copy only output to a smaller non-root JRE stage.

```
# syntax=docker/dockerfile:1
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY Demo.java Health.java ./
RUN mkdir /out && javac --release 17 --add-modules jdk.httpserver \
    -d /out Demo.java Health.java
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build --chown=10001:10001 /out/ /app/
ENV PORT=8080
USER 10001:10001
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=5s --start-period=15s --retries=3 \
    CMD ["java", "-cp", "/app", "Health"]
ENTRYPOINT ["java", "--add-modules", "jdk.httpserver", "-cp", "/app", "Demo"]
```

> Keep the exact files in one directory before running Lab 07. Build inputs, runtime files and health behavior are intentionally separate.

JAVA WORKED EXAMPLE / FILE 4 OF 4

.dockerignore

Keep the Docker build context small and avoid unnecessary local files.

```
*
!Demo.java
!Health.java
!Dockerfile
!.dockerignore
```

> Keep the exact files in one directory before running Lab 07. Build inputs, runtime files and health behavior are intentionally separate.

05 / HANDS-ON PRACTICE — LAB 01 / 12

L01 Create, start, inspect and remove

Observe that an exited process and its container object have separate lifetimes.

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
docker info >/dev/null
DK_NAME="dk-l01-$$"
! docker container inspect "$DK_NAME" >/dev/null 2>&1
trap 'docker rm -f "$DK_NAME" >/dev/null 2>&1 || true' EXIT
docker create --name "$DK_NAME" alpine:3.22 echo hello
docker start -a "$DK_NAME"
test "$(docker inspect -f '{{.State.Status}}' "$DK_NAME")" = exited
test "$(docker inspect -f '{{.State.ExitCode}}' "$DK_NAME")" = 0
test "$(docker logs "$DK_NAME")" = hello
docker start -a "$DK_NAME"
docker rm "$DK_NAME" >/dev/null
printf 'PASS: same container started twice, then removed\n'
```

Expected checkpoint: PASS: same container started twice, then removed\n'

05 / HANDS-ON PRACTICE — LAB 02 / 12

L02 Predict ENTRYPOINT and CMD arguments

Build a tiny argument-printing image and verify which default runtime arguments are replaced.

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
test -z "$(ls -A)"
DK_IMAGE="dk-l02:$$"
trap 'docker image rm "$DK_IMAGE" >/dev/null 2>&1 || true' EXIT
cat > Dockerfile <<'EOF'
FROM alpine:3.22
ENTRYPOINT ["printf", "%s\n"]
CMD ["default"]
EOF
docker build -t "$DK_IMAGE" .
test "$(docker run --rm "$DK_IMAGE")" = default
test "$(docker run --rm "$DK_IMAGE" custom)" = custom
test "$(docker run --rm --entrypoint echo "$DK_IMAGE" replaced)" = replaced
printf 'PASS: CMD replaced; ENTRYPOINT kept unless overridden\n'
```

Expected checkpoint: PASS: CMD replaced; ENTRYPOINT kept unless overridden\n'

05 / HANDS-ON PRACTICE — LAB 03 / 12

L03 Named volume persistence

Write data with one container, read it with another, and remove only the disposable lab volume.

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
DK_VOLUME="dk-l03-$$"
! docker volume inspect "$DK_VOLUME" >/dev/null 2>&1
docker volume create "$DK_VOLUME" >/dev/null
trap 'docker volume rm "$DK_VOLUME" >/dev/null 2>&1 || true' EXIT
docker run --rm \
  --mount "type=volume,src=$DK_VOLUME,dst=/data" \
  alpine:3.22 sh -c 'printf "retained\n" > /data/note'
DK_TEXT=$(docker run --rm \
  --mount "type=volume,src=$DK_VOLUME,dst=/data,readonly" \
  alpine:3.22 cat /data/note)
test "$DK_TEXT" = retained
docker volume rm "$DK_VOLUME" >/dev/null
printf 'PASS: data survived writer removal and was read again\n'
```

Expected checkpoint: PASS: data survived writer removal and was read again\n'

05 / HANDS-ON PRACTICE — LAB 04 / 12

L04 Read-only bind mount

Confirm a container can read a host file but cannot write through a read-only bind.

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
test -z "$(ls -A)"
mkdir fixture
printf 'host-value\n' > fixture/note
DK_DIR="$PWD/fixture"
test "$(docker run --rm \
  --mount "type=bind,src=$DK_DIR,dst=/config,readonly" \
  alpine:3.22 cat /config/note)" = host-value
if docker run --rm \
  --mount "type=bind,src=$DK_DIR,dst=/config,readonly" \
  alpine:3.22 sh -c 'echo changed > /config/note'; then
  printf 'FAIL: write unexpectedly succeeded\n' >&2
  exit 1
fi
test "$(cat fixture/note)" = host-value
printf 'PASS: read worked, write failed, host value unchanged\n'
```

Expected checkpoint: PASS: read worked, write failed, host value unchanged\n'

05 / HANDS-ON PRACTICE — LAB 05 / 12

L05 Service discovery on a user-defined network

Resolve a container name and fetch HTTP without publishing a host port.

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
DK_NET="dk-l05-net-$$"
DK_WEB="dk-l05-web-$$"
! docker network inspect "$DK_NET" >/dev/null 2>&1
! docker container inspect "$DK_WEB" >/dev/null 2>&1
docker network create "$DK_NET" >/dev/null
trap 'docker rm -f "$DK_WEB" >/dev/null 2>&1 || true;
      docker network rm "$DK_NET" >/dev/null 2>&1 || true' EXIT
docker run -d --name "$DK_WEB" --network "$DK_NET" nginx:stable
docker run --rm --network "$DK_NET" alpine:3.22 sh -ec '
  for n in 1 2 3 4 5 6 7 8 9 10; do
    if wget -qO /tmp/page "http://$1"; then
      grep -q "Welcome to nginx" /tmp/page
      exit 0
    fi
    sleep 1
  done
  exit 1
' sh "$DK_WEB"
printf 'PASS: peer HTTP worked through container-name DNS\n'
```

Expected checkpoint: PASS: peer HTTP worked through container-name DNS\n'

05 / HANDS-ON PRACTICE — LAB 06 / 12

L06 Publish HTTP only on host loopback

Use a random local host port and distinguish its value from the container listener port.

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
DK_NAME="dk-l06-$$"
! docker container inspect "$DK_NAME" >/dev/null 2>&1
trap 'docker rm -f "$DK_NAME" >/dev/null 2>&1 || true' EXIT
docker run -d --name "$DK_NAME" \
  -p 127.0.0.1::80 nginx:stable
DK_ENDPOINT=$(docker port "$DK_NAME" 80/tcp)
case "$DK_ENDPOINT" in 127.0.0.1:*) ;; *) exit 1 ;; esac
DK_READY=0
for DK_N in 1 2 3 4 5 6 7 8 9 10; do
  if curl -fsS "http://$DK_ENDPOINT" -o /dev/null; then
    DK_READY=1
    break
  fi
  sleep 1
done
test "$DK_READY" = 1
printf 'PASS: host loopback HTTP reached container port 80\n'
```

Expected checkpoint: PASS: host loopback HTTP reached container port 80\n'

05 / HANDS-ON PRACTICE — LAB 07 / 12

L07 Multi-stage non-root Java service

Use the complete Demo.java, Health.java, Dockerfile and .dockerignore from the worked example to

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
test -f Demo.java && test -f Health.java && test -f Dockerfile
DK_IMAGE="dk-l07:$$"
DK_NAME="dk-l07-$$"
! docker container inspect "$DK_NAME" >/dev/null 2>&1
trap 'docker rm -f "$DK_NAME" >/dev/null 2>&1 || true;
      docker image rm "$DK_IMAGE" >/dev/null 2>&1 || true' EXIT
docker build -t "$DK_IMAGE" .
test "$(docker image inspect -f '{{.Config.User}}' "$DK_IMAGE")" = 10001:10001
docker run -d --name "$DK_NAME" --memory=256m "$DK_IMAGE"
DK_READY=0
for DK_N in $(seq 1 30); do
  if docker exec "$DK_NAME" java -cp /app Health; then
    DK_READY=1
    break
  fi
  sleep 1
done
test "$DK_READY" = 1
docker stop --time 10 "$DK_NAME" >/dev/null
test "$(docker inspect -f '{{.State.OOMKilled}}' "$DK_NAME")" = false
printf 'PASS: non-root Java service answered its health probe\n'
```

> **Expected checkpoint:** PASS: non-root Java service answered its health probe\n'

05 / HANDS-ON PRACTICE — LAB 08 / 12

L08 Observe selective cache invalidation

Build twice unchanged, then change a source input and verify the resulting artifact changes. Inspect

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
test -z "$(ls -A)"
DK_IMAGE="dk-l08:$$"
trap 'docker image rm "$DK_IMAGE" >/dev/null 2>&1 || true' EXIT
printf 'dependency-v1\n' > dependencies.txt
printf 'source-v1\n' > source.txt
cat > Dockerfile <<'EOF'
FROM alpine:3.22
WORKDIR /app
COPY dependencies.txt .
RUN cp dependencies.txt installed.txt
COPY source.txt .
CMD ["cat", "/app/source.txt"]
EOF
docker build --progress=plain -t "$DK_IMAGE" . > first.log 2>&1
docker build --progress=plain -t "$DK_IMAGE" . > second.log 2>&1
test "$(docker run --rm "$DK_IMAGE")" = source-v1
printf 'source-v2\n' > source.txt
docker build --progress=plain -t "$DK_IMAGE" . > third.log 2>&1
test "$(docker run --rm "$DK_IMAGE")" = source-v2
printf 'PASS: changed source was packaged; review cache logs\n'
```

> **Expected checkpoint:** PASS: changed source was packaged; review cache logs\n'

05 / HANDS-ON PRACTICE — LAB 09 / 12

L09 Build secret mount without packaging the file

Use a fake, non-sensitive build credential to show temporary secret access and check the final

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
test -z "$(ls -A)"
DK_IMAGE="dk-l09:$$"
trap 'docker image rm "$DK_IMAGE" >/dev/null 2>&1 || true' EXIT
printf 'demonstration-only\n' > build-token.txt
printf 'build-token.txt\n' > .dockerignore
cat > Dockerfile <<'EOF'
# syntax=docker/dockerfile:1
FROM alpine:3.22
RUN --mount=type=secret,id=token,required=true \
    test -s /run/secrets/token && printf 'built\n' > /result
CMD ["cat", "/result"]
EOF
docker build --secret id=token,src=build-token.txt \
  -t "$DK_IMAGE" .
test "$(docker run --rm "$DK_IMAGE")" = built
docker run --rm "$DK_IMAGE" sh -ec \
  'test ! -e /run/secrets/token; test ! -e /build-token.txt'
printf 'PASS: build used mount; secret file absent at runtime\n'
```

Expected checkpoint: PASS: build used mount; secret file absent at runtime\n'

05 / HANDS-ON PRACTICE — LAB 10 / 12

L10 Unhealthy does not automatically mean restarted

Observe a live process becoming unhealthy while a standalone restart policy has no exit to act on.

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
DK_NAME="dk-l10-$$"
! docker container inspect "$DK_NAME" >/dev/null 2>&1
trap 'docker rm -f "$DK_NAME" >/dev/null 2>&1 || true' EXIT
docker run -d --name "$DK_NAME" --restart=on-failure \
  --health-cmd=false --health-interval=1s \
  --health-timeout=1s --health-retries=1 \
  alpine:3.22 sleep 120
DK_BAD=0
for DK_N in $(seq 1 30); do
  DK_STATE=$(docker inspect -f '{{.State.Health.Status}}' "$DK_NAME")
  if test "$DK_STATE" = unhealthy; then DK_BAD=1; break; fi
  sleep 1
done
test "$DK_BAD" = 1
test "$(docker inspect -f '{{.State.Running}}' "$DK_NAME")" = true
test "$(docker inspect -f '{{.RestartCount}}' "$DK_NAME")" = 0
printf 'PASS: unhealthy process remained running without restart\n'
```

Expected checkpoint: PASS: unhealthy process remained running without restart\n'

05 / HANDS-ON PRACTICE — LAB 11 / 12

L11 Inspect JVM sizing under resource limits

Check that the runtime object has the requested memory ceiling, then inspect JVM-detected resource

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
DK_NAME="dk-l11-$$"
! docker container inspect "$DK_NAME" >/dev/null 2>&1
trap 'docker rm -f "$DK_NAME" >/dev/null 2>&1 || true' EXIT
docker create --name "$DK_NAME" --memory=512m --cpus=1 \
  eclipse-temurin:21-jre \
  java -XX:MaxRAMPercentage=60 \
  -XshowSettings:system -XshowSettings:vm -version
docker start -a "$DK_NAME"
test "$(docker inspect -f '{{.HostConfig.Memory}}' "$DK_NAME")" = 536870912
test "$(docker inspect -f '{{.State.ExitCode}}' "$DK_NAME")" = 0
printf 'PASS: 512 MiB configured; inspect JVM sizing output\n'
```

> **Expected checkpoint:** PASS: 512 MiB configured; inspect JVM sizing output\n'

05 / HANDS-ON PRACTICE — LAB 12 / 12

L12 Compose database readiness and persistence

Start a disposable PostgreSQL 17 service with a file secret, write data, recreate the service, and

> **Run only in a disposable practice directory.** Needs a locally accessible Docker Engine. This script passed Bash syntax validation, but Docker runtime results were not executed here.

Lab script

```
set -eu
test -z "$(ls -A)"
mkdir secrets
printf 'disposable-lab-password\n' > secrets/db_password.txt
chmod 600 secrets/db_password.txt
cat > compose.yaml <<'EOF'
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_USER: app
      POSTGRES_DB: orders
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
    volumes: [data:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d orders"]
      interval: 2s
      timeout: 3s
      retries: 30
volumes:
  data:
secrets:
  db_password:
    file: ./secrets/db_password.txt
EOF
DK_PROJECT="dk-l12-$$"
trap 'docker compose -p "$DK_PROJECT" down -v >/dev/null 2>&1 || true' EXIT
docker compose -p "$DK_PROJECT" up -d --wait --wait-timeout 120
docker compose -p "$DK_PROJECT" exec -T db psql -v ON_ERROR_STOP=1 \
  -U app -d orders -c "CREATE TABLE note(v text); INSERT INTO note VALUES ('kept');"
docker compose -p "$DK_PROJECT" down
docker compose -p "$DK_PROJECT" up -d --wait --wait-timeout 120
DK_TEXT=$(docker compose -p "$DK_PROJECT" exec -T db \
  psql -U app -d orders -Atc 'SELECT v FROM note')
test "$DK_TEXT" = kept
printf 'PASS: healthy PostgreSQL retained data across recreation\n'
```

> **Expected checkpoint:** PASS: healthy PostgreSQL retained data across recreation\n'

06 / QUICK REVISION

Docker commands without confusion

`docker build -t app:local .`	Build an image from the current context
`docker run --rm image`	Create + start; auto-remove when exited
`docker ps -a`	Show both running and stopped containers
`docker logs --tail=100 name`	Read recorded stdout/stderr
`docker inspect name`	Inspect effective runtime configuration/state
`docker exec -it name sh`	New shell only if container is running and contains sh
`docker stop --time 45 name`	Give process time to shut down
`docker compose up -d`	Reconcile multi-service app
`docker compose down`	Remove Compose services/networks; not named volumes by default
`docker volume ls`	List managed volumes
`docker stats --no-stream`	Snapshot of resource use
`docker context show`	Check which engine receives commands

> Read-only observation first. Commands such as `rm -f`, `volume rm`, or `compose down -v` can remove state; use only with explicit understanding.

06 / QUICK REVISION

Six distinctions to say clearly

Image vs container	Image is the packaged template; container is the created instance with process and writable state.
EXPOSE vs -p	EXPOSE declares intent; -p publishes a host port. Neither makes a dead application listen.
Volumes vs backups	Volume persists locally; backup must be restorable, ideally after losing the original host.
Health vs restart	Unhealthy status does not itself guarantee a restart under Docker's restart policy.
Tags vs digests	Tag is a movable name; digest is content identity. Identity alone does not prove trust.
Heap vs container memory	Heap is only part of JVM memory; thread stacks, metaspace and native buffers count too.

A safe production checklist

Tested image digest · non-root process · least-privilege policy · bound CPU/memory · health endpoints · graceful shutdown · protected secrets · volume backups · observability · rollback plan.

07 / YOUR PRACTICE ROUTE

14-day interview sprint

Study the short answers, draw the diagrams, then practice the commands in disposable environments.

DAY 01	Docker, images, containers	Q001–Q005; describe kernel boundary
DAY 02	Engine internals and lifecycle	Q006–Q015; inspect a disposable container
DAY 03	Registries and image identity	Q016–Q020; distinguish tags/digests
DAY 04	Dockerfile fundamentals	Q021–Q030; explain ENTRYPOINT and SIGTERM
DAY 05	Build caching and multi-stage	Q031–Q040; build Java source
DAY 06	Volumes and networks	Q041–Q050; run Labs 03–06
DAY 07	Compose and configuration	Q051–Q060; configure a database
DAY 08	Health and shutdown	Q061–Q065; simulate a failed probe
DAY 09	JVM memory, logs and evidence	Q066–Q075; inspect limits
DAY 10	Security and supply chain	Q076–Q085; scan and review an image
DAY 11	CI, digests and architectures	Q086–Q095; plan multi-platform builds
DAY 12	Stateful apps and orchestration	Q096–Q105; design recovery
DAY 13	Incident responses and labs	Q106–Q110; finish remaining labs
DAY 14	Two mock interviews	Explain five diagrams without notes

The schedule is guidance, not a measured guarantee of interview results. Prioritize failure scenarios and explain what every command changes.

08 / PRIMARY REFERENCES

Official reference directory

References retained from the uploaded Docker guide. Use the appropriate documentation for your installed Docker, JDK and Spring versions.

D01

Docker platform overview

https://docs.docker.com/get-started/docker-overview/

D02

Running containers and runtime options

https://docs.docker.com/engine/containers/run/

D03

Dockerfile instruction reference

https://docs.docker.com/reference/dockerfile/

D04

Build context and ignore rules

https://docs.docker.com/build/concepts/context/

D05

Image-build practices

https://docs.docker.com/build/building/best-practices/

D06

Multi-stage builds

https://docs.docker.com/build/building/multi-stage/

D07

Cache invalidation

https://docs.docker.com/build/cache/invalidation/

D08

Cache optimization

https://docs.docker.com/build/cache/optimize/

D09

External cache backends

https://docs.docker.com/build/cache/backends/

08 / PRIMARY REFERENCES

Official reference directory

References retained from the uploaded Docker guide. Use the appropriate documentation for your installed Docker, JDK and Spring versions.

D10

Build secret mounts

https://docs.docker.com/build/building/secrets/

D11

Named volumes

https://docs.docker.com/engine/storage/volumes/

D12

Bind mount semantics

https://docs.docker.com/engine/storage/bind-mounts/

D13

Linux tmpfs mounts

https://docs.docker.com/engine/storage/tmpfs/

D14

Network overview and DNS

https://docs.docker.com/engine/network/

D15

User-defined bridge networks

https://docs.docker.com/engine/network/drivers/bridge/

D16

Port publication and loopback caveat

https://docs.docker.com/engine/network/port-publishing/

D17

Host-network platform support

https://docs.docker.com/engine/network/drivers/host/

D18

CPU and memory constraints

https://docs.docker.com/engine/containers/resource_constraints/

08 / PRIMARY REFERENCES

Official reference directory

References retained from the uploaded Docker guide. Use the appropriate documentation for your installed Docker, JDK and Spring versions.

D19

Standalone restart policies

https://docs.docker.com/engine/containers/start-containers-automatically/

D20

Stop signals and timeout

https://docs.docker.com/reference/cli/docker/container/stop/

D21

Additional processes with exec

https://docs.docker.com/reference/cli/docker/container/exec/

D22

Container inspection

https://docs.docker.com/reference/cli/docker/container/inspect/

D23

Container file copying

https://docs.docker.com/reference/cli/docker/container/cp/

D24

Resource statistics and memory display

https://docs.docker.com/reference/cli/docker/container/stats/

D25

Docker storage usage

https://docs.docker.com/reference/cli/docker/system/df/

D26

System prune categories

https://docs.docker.com/reference/cli/docker/system/prune/

D27

Image pull and digest references

https://docs.docker.com/reference/cli/docker/image/pull/

08 / PRIMARY REFERENCES

Official reference directory

References retained from the uploaded Docker guide. Use the appropriate documentation for your installed Docker, JDK and Spring versions.

D28

Image tags and repository names

https://docs.docker.com/reference/cli/docker/image/tag/

D29

Image history

https://docs.docker.com/reference/cli/docker/image/history/

D30

Image archives with save

https://docs.docker.com/reference/cli/docker/image/save/

D31

Container filesystem export limits

https://docs.docker.com/reference/cli/docker/container/export/

D32

Engine contexts

https://docs.docker.com/reference/cli/docker/context/

D33

Logging drivers and retention

https://docs.docker.com/engine/logging/configure/

D34

Engine security model

https://docs.docker.com/engine/security/

D35

Rootless engine operation

https://docs.docker.com/engine/security/rootless/

D36

UID namespace remapping

https://docs.docker.com/engine/security/userns-remap/

08 / PRIMARY REFERENCES

Official reference directory

References retained from the uploaded Docker guide. Use the appropriate documentation for your installed Docker, JDK and Spring versions.

D37

Seccomp policy

https://docs.docker.com/engine/security/seccomp/

D38

Compose service configuration

https://docs.docker.com/reference/compose-file/services/

D39

Compose startup and shutdown order

https://docs.docker.com/compose/how-tos/startup-order/

D40

Compose network discovery

https://docs.docker.com/compose/how-tos/networking/

D41

Local Compose secrets

https://docs.docker.com/compose/how-tos/use-secrets/

D42

Multi-platform build strategies

https://docs.docker.com/build/building/multi-platform/

D43

Build variables and platform arguments

https://docs.docker.com/build/building/variables/

D44

buildx build flags and exports

https://docs.docker.com/reference/cli/docker/buildx/build/

D45

Inspecting image indexes

https://docs.docker.com/reference/cli/docker/buildx/imagetools/inspect/

08 / PRIMARY REFERENCES

Official reference directory

References retained from the uploaded Docker guide. Use the appropriate documentation for your installed Docker, JDK and Spring versions.

D46

Base image dependencies

https://docs.docker.com/build/building/base-images/

D47

Build attestation concepts

https://docs.docker.com/build/metadata/attestations/

D48

SBOM attestations

https://docs.docker.com/build/metadata/attestations/sbom/

D49

Build provenance

https://docs.docker.com/build/metadata/attestations/slsa-provenance/

D50

Docker build integration in CI

https://docs.docker.com/build/ci/github-actions/

D51

Swarm orchestration

https://docs.docker.com/engine/swarm/

J01

Java 21 runtime flags and container detection

https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html

S01

Spring Boot Dockerfile guidance

https://docs.spring.io/spring-boot/reference/packaging/container-images/dockerfiles.html

S02

Spring Boot graceful shutdown

https://docs.spring.io/spring-boot/reference/web/graceful-shutdown.html

08 / PRIMARY REFERENCES

Official reference directory

References retained from the uploaded Docker guide. Use the appropriate documentation for your installed Docker, JDK and Spring versions.

S03

Spring Boot 3.5 external config and config trees

https://docs.spring.io/spring-boot/3.5/reference/features/external-config.html

S04

Spring Boot Maven image building

https://docs.spring.io/spring-boot/3.5/maven-plugin/build-image.html

O01

OCI specifications and responsibilities

https://opencontainers.org/about/overview/

K01

Kubernetes CRI runtime integration

https://kubernetes.io/docs/setup/production-environment/container-runtimes/

K02

Kubernetes desired-state deployments

https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

P01

Official PostgreSQL image contract

https://hub.docker.com/_/postgres

P02

PostgreSQL 17 logical backups and restore

https://www.postgresql.org/docs/17/backup-dump.html

A01

Apache Kafka broker listener configuration

https://kafka.apache.org/42/configuration/broker-configs/